Atomic Visibility Is the Price of Mutable Redundancy: Schema, Placement and Write Order on Sharded SQL Engines

Yanni Tang
University of Auckland
Auckland, New Zealand
ytan370@aucklanduni.ac.nz

Zhuoxing Zhang*
University of Auckland
Auckland, New Zealand
zhuoxing.zhang@auckland.ac.nz

ABSTRACT

Sharded SQL engines such as Citus and Vitess commit each shard separately without a global snapshot, so plain SQL readers may see illegal intermediate states even under two-phase commit. Atomic visibility hides these states but costs global coordination or a rewrite of every reader. Ordering the writes needs neither, yet does not always help. We initiate research on *order-sufficiency*, the existence of a legal write order for every modification given the immutable attributes of a workload. Our central result shows that, under functional dependencies (FDs) and foreign keys with cached columns, an update that writes new values needs atomic visibility exactly when it rewrites a value and its copy. For arbitrary modifications, order-sufficiency is undecidable, but polynomial for FDs and three classes of inclusion dependencies. Attributes become immutable once their modifications are rewritten as deletions and insertions, and choosing them is NP-hard but a minimum cut on forests of foreign keys. They decide the shard keys that confine atomic visibility to one shard, and the other updates compile to leveled commits, which go straight to the shards and take less time than two-phase commit. In GitLab, Sentry and ten further applications, most multi-table writes need only order, and almost all that need atomic visibility copy or derive mutable attributes. On Vitess, and on Citus once a worker lags a millisecond, one two-phase commit exposes orphaned children, which leveled commits hide from readers that read in reverse write order or finish within a pause between commits. In summary, atomic visibility is the price of mutable redundancy.

PVLDB Reference Format:
Yanni Tang and Zhuoxing Zhang. Atomic Visibility Is the Price of Mutable Redundancy: Schema, Placement and Write Order on Sharded SQL Engines. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/zhuoxingzhang/order-sufficiency.

1 INTRODUCTION

Sharded SQL engines such as Citus [23] and Vitess [66] scale relational applications by partitioning every table into shards. A transaction that writes several shards commits by two-phase commit (2PC), but each shard makes its writes visible at its own commit point. No global snapshot hides the gaps between these commit points, so 2PC makes such a transaction atomic but not *atomically visible*. All its shards commit or none, but a reader may see the writes of one shard and not those of another. Plain SQL readers do see such intermediate states. On Vitess, for example, a reader that saw a new child found no parent in 86% of the runs whose 2PC wrote the child first (Sec. 6.3). Applications that split their data across databases, such as GitLab and Sentry, commit in each database separately and expose the same states.

Whether these states harm an application depends on the constraints that its schema promises. The engine checks keys and foreign keys within a shard, but the application also relies on functional dependencies (FDs) and on inclusion dependencies (INDs) across shards, such as foreign keys and copies of attributes. States that violate them do harm in practice. In GitLab, a background task removes the children of a deleted parent in other databases. A migration that read such a dangling child, an intermediate state of the deletion, aborted an upgrade in a severity-one incident [32]. An update whose intermediate states may violate these constraints needs atomic visibility, unless its writes can be committed one at a time so that every state it passes through is legal.

Providing atomic visibility is costly. Engines with global snapshots, such as Spanner [21] and CockroachDB [62], provide it, but every distributed transaction then pays for global coordination [12]. RAMP transactions [5, 6], as deployed in RAMP-TAO [19], add it on top of 2PC, but they rewrite every reader and attach metadata to every read. Highly available transactions [2] and causal consistency [7, 43, 44] likewise rewrite every reader, and their metadata is at best of constant size [45]. Indeed, atomically visible reads must wait or read stale data [64]. We keep the readers as they are: plain SQL statements of object-relational mappers, reports and other services. The order of the writes alone must then keep readers from seeing illegal states. Whether such an order exists depends on the update, as the following example shows.

Example 1.1 (Groups and projects). Relation $Group(gid, path)$ stores groups with their URL path, and relation $Project(pid, gid, gpath)$ stores projects with their group and a cached copy of the group's path, much like the routes of GitLab [33] (Sec. 6.2). The constraints are a foreign key and an IND that keeps the cached copy correct:

$$Project[gid] \subseteq Group[gid],$$
$$Project[gid, gpath] \subseteq Group[gid, path].$$

Groups are sharded by *gid* and projects by *pid*, and the workload creates groups and projects, moves projects between groups, and renames groups. Creating a group with its first project writes two

*Corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

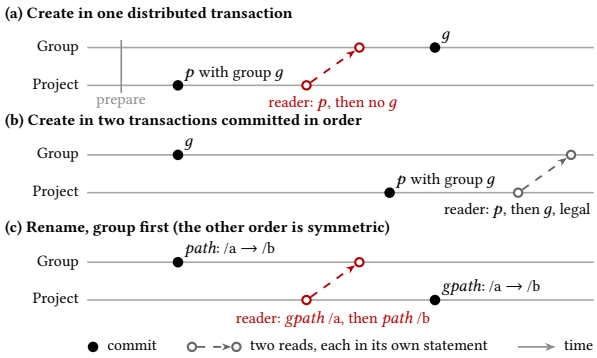


Figure 1: Commits and Reads for the Updates of Ex. 1.1

shards. If both writes commit in one distributed transaction, a reader may see the project before its group. If the application commits the group first, a reader that reads the project and then its group sees no orphan. Hence, for creation, the order of the writes and not their atomicity decides which states readers see. Renaming a group changes the group and every one of its projects. Whichever is written first, a reader may see a project whose cached path matches no group, so no order helps.

The example yields three observations, the first two of which Fig. 1 illustrates on two shards. Firstly, for readers that read a project before its group, write order prevents an anomaly that 2PC does not. For insertions and deletions, such an order follows the classical rule, which object-relational mappers apply: write referenced entities first and delete them last. Secondly, some modifications, which change attributes of existing entities, admit no legal order, and no rule tells which. This depends on the schema and the workload: without the cached copy, or if groups were never renamed, every modification of Ex. 1.1 would be harmless in any order. Thirdly, placement can substitute for atomic visibility: if projects never move and lie on the shard of their group, a rename touches one shard and becomes visible at once. This raises two questions.

(Q1) Which updates of a workload need atomic visibility because no write order keeps the constraints satisfied, and can this be decided?

(Q2) How should a designer choose which attributes the workload may modify, the shard keys and the write orders, so that fewer updates need atomic visibility and the others stay legal after every commit?

Previous work does not answer these questions. Invariant confluence [3] decides whether concurrent operations need coordination, not whether the writes of one update can be ordered so that readers see only legal states. Transaction chopping [59] and transaction chains [72] commit the pieces of a transaction in order, but they analyze all transactions, readers included, whereas the readers of a sharded engine are arbitrary SQL statements. We are unaware of work that characterizes when write order alone keeps the constraints of a schema satisfied.

We treat *order-sufficiency*, the existence of a legal write order for every modification, as a property of the schema. As the second observation suggests, it is relative to the attributes that a workload never modifies, which we call *frozen*. Fig. 2 contrasts the three cases that our results separate: updates that need no order, updates for which write order suffices, and updates for which it is not enough.

Our central result locates the need for atomic visibility in *mutable redundancy*. It concerns *fresh* updates, which write values that no row held before, as renames and new tokens do. Under FDs and foreign keys with cached columns, such an update needs atomic visibility exactly when it rewrites a value and its copy (Thm. 4.6). Under the same constraints, write order adds nothing to modifications that may also exchange values, such as two users swapping their email addresses. That is, either every modification is legal in any order, or some modification needs atomic visibility. Classical normal forms emerge as special cases. Under FDs, every fresh update of every workload is legal in any order whenever the schema is in Boyce-Codd normal form. Every modification outside the primary keys is legal in any order exactly when the schema is in the object normal form of Biskup [13]. Write order matters instead for insertions, deletions and mixed updates, such as *re-keying* a group. Re-keying inserts a copy of the group under a new key value, moves its projects to the copy and deletes the old group. Write order also matters for one-way conditions on a status, such as an unarchived project needing an unarchived group.

The central result suggests a design in three steps. Freezing removes mutable redundancy by turning modifications into re-keying, shard keys keep the modifications that still need atomic visibility within one shard, and leveled commits write every other update in a legal order. They commit single-shard transactions in parallel rounds, adding a round only where the order crosses shards.

Real applications confirm that write order is the common case and mutable redundancy the exception. We call the writes of one code location a *template* (Sec. 2.3). In GitLab [33] and Sentry [58], 76 of the 97 templates that write across databases need only order. Every template that needs atomic visibility maintains a copy or a derived value of a mutable attribute, such as a full path in GitLab. In ten further applications, hand labels find no other reason for atomic visibility. However, write order protects only readers that read first what an update writes last, which suffices for every reader of 47 of the 76 templates. A pause between the rounds of leveled commits also protects the other readers that finish within it.

We summarize our contributions as follows.

(1) Conceptually, we introduce order-sufficiency, where some write order keeps every state legal, and *order-insensitivity*, where every order does, both relative to the frozen attributes. For any FDs and INDs, a schema is order-insensitive whenever it follows from a syntactic set of constraints that no modification can violate. For FDs, this condition is also necessary (Secs. 2 and 3).

(2) Theoretically, we prove the central result for FDs and for a class of acyclic foreign keys with cached columns. For arbitrary modifications, order-sufficiency is undecidable for FDs and INDs. On unary INDs, on acyclic INDs that reference keys and on this class, it is decidable in polynomial time and coincides with order-insensitivity, as for FDs. Conditional INDs of an acyclic class, which keep one-way conditions on a status, give every update a legal order (Sec. 4).

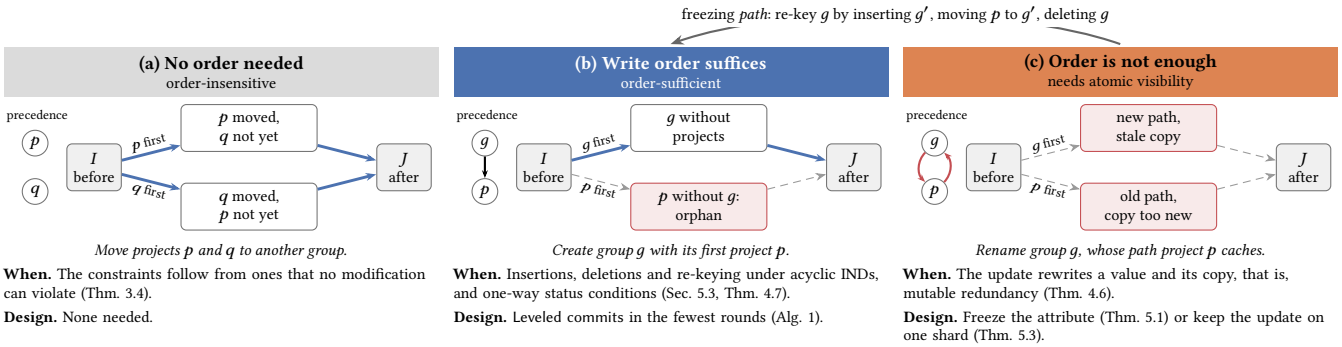


Figure 2: When Updates Need No Order, Write Order or Atomic Visibility. An arrow from g to p in a precedence graph means that p may become visible only with or after g, and red arrows form a cycle. Blue arrows mark legal write orders, and dashed arrows orders that pass an illegal intermediate state (red box).

(3) Algorithmically, we show that choosing the attributes to freeze is NP-hard but a minimum cut when foreign keys with cached columns form a forest. Under FDs, shard keys let every modification run in legal single-shard steps exactly when the frozen attributes of every FD that modifications can violate determine the shard key. Under foreign keys with cached columns, they do so exactly when every reference whose updates need atomic visibility stores the shard key of the entity that it references. Write orders compile to leveled commits, which need the fewest rounds, protect every reader that finishes within a pause between rounds, and after a crash complete or stop in a legal state without lost updates (Sec. 5).

(4) Experimentally, we find that one 2PC exposes orphans on Vitess, and on Citus once one worker commits 1 ms late, whereas leveled commits expose none to the readers that write order protects. Sent straight to the shards, leveled commits also take less time than 2PC for up to six rounds. The declared shard keys of GitLab co-locate every reference that needs atomic visibility with the row that it references, as long as these keys stay frozen. The more skewed the update frequencies, the more an optimal freeze saves over freezing every copied source (Sec. 6).

In summary, for fresh updates atomic visibility is the price of mutable redundancy, which freezing and placement can remove, while write order keeps every other update legal after each commit.

Organization. Sec. 2 fixes the engine model and defines order-sufficiency. Secs. 3 and 4 answer (Q1) for FDs and INDs, with the central result in Sec. 4.2, and Sec. 5 answers (Q2) by freezing attributes, choosing shard keys and compiling write orders. Sec. 6 tests these answers on GitLab, Sentry and ten further applications and on Citus and Vitess. Sec. 7 discusses related work, and Sec. 8 concludes. *The proofs and further results are in the full version,* ***order-sufficiency-full.pdf*** *in our artifact repository.*

2 MODEL AND PROBLEM

To answer (Q1), we first fix what readers of a sharded SQL engine can observe, and then define order-sufficiency.

2.1 Sharded SQL Engines

A sharded SQL engine partitions every table into shards by a shard key and stores each shard on one node. In 2PC, all shards prepare, and each shard then makes the writes visible at its own commit point. The engine guarantees no order among these commit points and keeps no global snapshot. Citus [23] routes statements through a coordinator node to the worker nodes of the shards and sends the second phase to all shards in parallel. Vitess [66], in its 2PC mode [67], commits the shard that it touched first, then the others. Both engines expose intermediate states (Sec. 6.3).

Whether an intermediate state harms a reader depends on how the reader reads, on what storage holds and on which constraints the engine checks. Our results hold under five assumptions.

(1) *Readers* issue plain SQL statements that are not rewritten. Each statement reads, on each shard, the latest state that the shard has committed, and it sees no version metadata. These are the minimal-delay readers of Tomsic et al. [64].

(2) *Storage* holds the logical relations, or projections of them, as physical tables, without version columns or commit flags that would let readers hide partial writes.

(3) *Constraints* declared as primary keys, unique constraints and foreign keys within a shard are checked by the engine. Other constraints, such as foreign keys across shards and functional dependencies, are relied upon by readers but not checked.

(4) *Isolation* is native: every shard runs multiversion concurrency control, and statements read only committed data.

(5) *Read order*: write order protects only readers that read the changed entities of an update in the reverse of their write order, each in a later statement. That is, such a reader reads first what the update writes last. Other readers are protected only by updates whose every intermediate state is legal (Sec. 2.3), or by pauses between commits that outlast their reads (Sec. 5.3).

Both engines meet assumptions 1 and 4 by default. Citus runs at read committed, and Vitess runs MySQL at repeatable read on every shard, where a statement outside a transaction reads the latest committed state [66]. Whether a read path meets assumption 5 depends on its direction. Following a foreign key from a child to its parent reads an insertion in reverse write order but a deletion in

write order, since an insertion writes the parent first and a deletion removes it last. Listing the children of a parent does the converse. In the applications of Sec. 6, which allow no joins across databases, every reader reads one database per statement, so its direction alone decides whether write order protects it.

2.2 Schemas, Workloads and Modification Updates

A schema consists of relations, each with a primary key $PK(R)$, and a set Σ of FDs and INDs with their standard meaning [1]. An IND $R[X] \subseteq S[Y]$ requires every tuple t of R to have a *witness*, a tuple s of S with $s[Y] = t[X]$. INDs may be composite and cyclic, and instances are finite. An *entity* is a relation together with a value of its primary key. A workload enters our results only through the attributes that it never modifies, which we call *frozen*. We write $Fz(R)$ for the frozen attributes of a relation R, which always include $PK(R)$. The other attributes are *mutable*, and a choice of Fz is a *freeze*. Where no confusion arises, Fz also denotes the set of all frozen attributes, qualified by their relations, and similarly for primary and shard keys.

An update of the workload leads from an instance I to an instance J and changes several entities. A reader may see any mix of their old and new tuples. This motivates the following definition.

Definition 2.1 (Modification update). A *modification update* is a pair (I, J) of instances that both satisfy Σ, contain the same entities, and agree on the frozen attributes of every entity. Its *changed entities* Ch are those whose tuples differ in I and J. For $T \subseteq Ch$, the *intermediate state* $I \oplus T$ takes the tuples of J for the entities in T and the tuples of I for all other entities.

The intermediate state $I \oplus T$ is what a reader sees when the writes to the entities in T are visible and the others are not. For example, renaming a group of Ex. 1.1 with two projects changes three entities. One of its intermediate states shows the new path of the group next to the old cached paths of the projects. Secs. 3 and 4 study modification updates, since updates that only insert or only delete follow the classical rule (Prop. 5.4). Thm. 4.7 and Sec. 5.3 extend the results to updates that also insert or delete entities.

2.3 Order-Sufficiency

An application can restrict the mixes that readers see by writing the changed entities one at a time, committing each write before it issues the next. It writes every entity once, with its new tuple, since readers would also see any temporary value. The storage then passes only through the prefix states $I \oplus \{e_1, \ldots, e_k\}$ of this order. Two notions arise: some order keeps every prefix state legal, or every intermediate state is legal.

Definition 2.2 (Order-sufficiency). A modification update is *executable* if some permutation $e_1, \ldots, e_n$ of Ch makes every prefix state $I \oplus \{e_1, \ldots, e_k\}$ satisfy Σ, and it is *order-insensitive* if every intermediate state $I \oplus T$ satisfies Σ. The schema Σ is *order-sufficient* under Fz if every modification update is executable, and *order-insensitive* under Fz if every modification update is order-insensitive.

Every prefix state is an intermediate state, so order-insensitivity implies order-sufficiency. We fix the language of orders as follows.

Definition 2.3 (Legal orders). An instance is *legal* if it satisfies Σ, and a permutation of Ch is a *legal order* if all its prefix states are legal. A modification update *needs atomic visibility* if it is not executable. Two changed entities *must become visible together* if every intermediate state that contains the new tuple of exactly one of them is illegal.

In Ex. 1.1, every move is executable, whereas a rename of a group with projects is not, since the group and its projects must become visible together (Ex. A.2). These notions carry over to the code locations that issue updates. A *template* names the relations into which a code location inserts, those from which it deletes and the attributes that it modifies, and its updates are those that change entities accordingly. It *needs atomic visibility* if one of its updates does, is *unconstrained* if no constraint relates two entities that it changes, and *needs only order* otherwise.

Order-sufficiency and order-insensitivity differ in which readers they protect. A reader that follows assumption 5, reading the changed entities in the reverse of the write order, observes a prefix state. Indeed, if it sees the new version of e_j, then every e_i with $i < j$ was committed before e_j and is read after it, so the reader also sees the new version of e_i. A reader that cannot choose its read order, such as a single join over several shards, can observe any intermediate state. Order-sufficiency is what write order supplies to the readers of assumption 5, and order-insensitivity is what an arbitrary reader needs. Neither notion is an isolation level. Both promise a reader neither a snapshot nor serializability, only that every state it observes satisfies Σ. Sec. 5.3 extends this promise to readers that finish within a pause between commits.

As Ex. 1.1 shows, order-sufficiency depends on the schema and on the attributes that the workload modifies. The decision problem behind (Q1) asks, given Σ and Fz, whether Σ is order-sufficient under Fz. Secs. 3 and 4 decide both notions for FDs and for three classes of INDs, and for fresh updates under FDs and foreign keys with cached columns. They also show that a class of conditional INDs gives every update a legal order. The design problems behind (Q2) ask which attributes to freeze and which shard keys to choose so that the schema becomes order-sufficient, or every update executable on its shards (Sec. 5).

3 CHARACTERIZING ORDER-SUFFICIENCY

Order-sufficiency is governed by the constraints that no modification of the workload can violate, since they hold in every intermediate state. This section identifies these constraints, shows that they guarantee order-insensitivity whenever they imply the schema, and shows that this test is exact for FDs. Throughout, implication means finite implication: a set Γ of constraints implies Σ, written $\Gamma \models \Sigma$, if every finite instance that satisfies Γ satisfies Σ. The closure Σ^+ is the set of FDs and INDs that Σ implies. For attributes X of a relation R, the closure X^+ is the set of attributes A of R with $\Sigma \models X \to A$.

3.1 Update-Stable Consequences

Freezing an attribute protects more than the attribute itself. Intuitively, consider an IND that copies attributes from a referenced entity identified by frozen values. The referenced entity is then the same before and after the update, and so are the copies of its frozen attributes. This motivates the following closure.

Definition 3.1 (Frozen closure). The *frozen closure* M^* is the least family of attribute sets such that $M^*(R) \supseteq Fz(R)$ for every relation R and that is closed under the following rule for every IND $R[X] \subseteq S[Y]$ in Σ^+. If Y contains a key K of S with $K \subseteq M^*(S)$, and the attributes of X at the positions of K belong to $M^*(R)$, then $X_i \in M^*(R)$ for every position i with $Y_i \in M^*(S)$.

That is, the closure adds every attribute that copies a frozen attribute of an entity identified by frozen values. For example, it adds *gpath* once *gid* and *path* are frozen. Its significance lies in the following invariant.

Lemma 3.2 (Invariance of the frozen closure). *Every modification update leaves the attributes in M^* of every entity unchanged.*

Constraints that involve only such unchanged values or frozen keys survive every modification update.

Definition 3.3 (Update-stable consequences). The set $\mathcal{G}$ of *update-stable consequences* contains (i) every FD $X \to A$ in Σ^+ such that X contains a key inside M^* or $X \cup \{A\} \subseteq M^*$, and (ii) every IND $R[X] \subseteq S[Y]$ in Σ^+ such that $Y \subseteq M^*(S)$, or such that $X \subseteq M^*(R)$ and Y contains a key of S inside $M^*(S)$.

Each constraint in $\mathcal{G}$ holds in every intermediate state of every modification update. Indeed, a violation would involve only values that the update does not change, by Lem. 3.2, or two tuples that would already violate a frozen key in I. In Ex. 1.1, $\mathcal{G}$ contains the foreign key but not the cached-path IND, exactly the constraint that a rename violates (Ex. A.3).

3.2 Sufficiency

Every constraint that $\mathcal{G}$ implies then holds in every intermediate state as well, whatever the FDs and INDs.

Theorem 3.4 (Sufficiency). *Let Σ be any set of FDs and INDs, cyclic INDs included. If $\mathcal{G} \models \Sigma$, then Σ is order-insensitive, and hence order-sufficient, under Fz.*

Thm. 3.4 reduces a question about all updates of a workload to an implication problem over the schema. Where order-sufficiency also implies $\mathcal{G} \models \Sigma$, order-sufficiency and order-insensitivity coincide and reduce to this implication. This direction fails for some schemas with INDs (Sec. 4.3) but holds for FDs.

3.3 Functional Dependencies

For FDs, the condition of Thm. 3.4 is also necessary and takes a syntactic form. Following Zaniolo [71], an FD $X \to A$ is *elementary* if $A \notin X$ and no proper subset of X determines A.

Theorem 3.5 (Exactness for FDs). *Let Σ contain only FDs. The following are equivalent: (1) Σ is order-sufficient under Fz; (2) Σ is order-insensitive under Fz; (3) $\mathcal{G} \models \Sigma$; (4) every elementary FD $X \to A$ in Σ^+ satisfies $X \subseteq Fz$, and also $A \in Fz$ if X is not a superkey. Condition (3) is decidable in polynomial time.*

The proof in App. A.2 yields more than the equivalence. With FDs only, either every modification update is order-insensitive, or some update has two changed entities that must become visible together. We illustrate this dichotomy on a variant of our running example.

Example 3.6 (Paths stored in projects). Suppose the path is stored only in *Project*, under the FD $gid \to gpath$, and consider *Project* on its own. Condition (4) fails because *gid* is not a superkey and *gpath* is mutable. Indeed, any two projects of a renamed group must become visible together: whichever project is written first, the intermediate state gives the two projects different paths. No order helps, and this design needs atomic visibility for every rename.

Condition (4) relates to normal forms, but it depends on the workload through *Fz*. For a schema in Boyce-Codd normal form (BCNF), it holds exactly when the workload freezes every minimal key. With only the primary key frozen, condition (4) forces the primary key to be the only minimal key and coincides with the object normal form of Biskup [13].

This requirement is strict. A relation of users in BCNF with a unique email address already violates it once users may change their address. The updates that prove the necessity part of Thm. 3.5 exchange values between two entities whenever a key or a left side is mutable, such as two users who swap their addresses. Applications rarely do so, since renames, new email addresses and rotated tokens write values that no other row holds. We call a modification update *fresh* if the values that it writes into left sides of FDs and right sides of INDs in Σ are new in their columns. Fresh updates relax condition (4) as follows.

Proposition 3.7 (Fresh updates). *Let Σ contain only FDs. Every fresh modification update is order-insensitive under Fz if and only if every elementary FD $X \to A$ in Σ^+ whose left side X is not a superkey has $A \in Fz$. Otherwise, some fresh update with two changed entities is not executable.*

Hence fresh updates need atomic visibility exactly where the schema stores a mutable attribute redundantly. Indeed, an FD $X \to A$ whose left side is not a superkey repeats an A-value in every tuple with the same X-value. In particular, every schema in BCNF is order-insensitive for fresh updates under every freeze, and users may change their email addresses after all. However, the design of Ex. 3.6 still needs atomic visibility, since a rename writes its new path into every project of the group. Thm. 4.6 extends this result to foreign keys with cached columns.

4 INCLUSION DEPENDENCIES

With INDs, the condition of Thm. 3.4 remains sufficient but is no longer necessary. Order-sufficiency even becomes undecidable, since finite implication of an FD by FDs and INDs is undecidable [17, 49], [1, Thm. 9.2.4]. Indeed, implication reduces to order-sufficiency and to order-insensitivity even when the workload modifies a single attribute (Prop. A.6). The difficulty lies in implication, not in write order, so a decidable test must restrict the INDs. Fig. 3 summarizes the results: three classes on which the condition is exact (Sec. 4.1), fresh updates on the third (Sec. 4.2), and conditional INDs, which give every update a legal order (Sec. 4.3).

4.1 Exact Classes

These classes share one conclusion with FDs, which we state once. For a schema Σ and a freeze *Fz*, consider the conditions (1) Σ is order-sufficient under *Fz*, (2) Σ is order-insensitive under *Fz*, and (3) $\mathcal{G} \models \Sigma$, as in Thm. 3.5. On each class below, they are equivalent

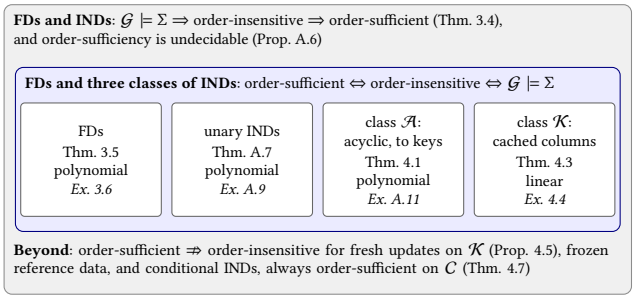


Figure 3: Order-Sufficiency by Class of Constraints

to a syntactic condition (4) that is decidable in polynomial time, so the condition of Thm. 3.4 is exact and write order adds nothing to modifications. The classes differ in how INDs interact with FDs.

Unary INDs. The first class allows only INDs between single attributes, which Cosmadakis et al. [22] call *unary*. They interact with FDs through cycles, which can make an attribute a key in every finite instance. Finite implication captures these cycles and decides order-sufficiency on this class in polynomial time (Thm. A.7).

Key-referencing INDs. The second class captures foreign keys that reference keys. The *IND graph* of Σ links R to S for every IND $R[X] \subseteq S[Y]$ in Σ. Let $\mathcal{A}$ contain the schemas whose IND graph is acyclic and whose INDs reference keys. That is, for every IND $R[X] \subseteq S[Y]$, the attributes Y form a minimal key of S under the FDs declared on S, and no relation is referenced through two different keys. In particular, acyclic foreign keys to primary keys yield schemas in $\mathcal{A}$. For these schemas, the FDs of each relation alone decide order-sufficiency.

Theorem 4.1 (Exactness for key-referencing INDs). *Let Σ belong to $\mathcal{A}$. Conditions (1) to (3) are equivalent to (4) for every relation R, the set F_R of FDs declared on R satisfies condition (4) of Thm. 3.5 in place of Σ, and to (5) every modification update with exactly two changed entities is executable. Condition (4) is decidable in polynomial time by Thm. 3.5.*

Foreign keys to primary keys thus never force atomic visibility on modifications by themselves, whereas the cached-path IND of Ex. 1.1 references no minimal key and lies outside $\mathcal{A}$ (Ex. A.11).

Foreign keys with cached columns. The third class adds what $\mathcal{A}$ excludes, namely copies of referenced attributes. An IND $R[PD] \subseteq S[LC]$ follows a pointer P to the primary key L of S and keeps every copy D_i equal to its source C_i. In Ex. 1.1, the pointer is *gid*, the copy *gpath* and the source *path*.

Definition 4.2 (Class $\mathcal{K}$). A schema belongs to the class $\mathcal{K}$ if its only FDs are the primary keys, its IND graph is acyclic, and every IND has the form $R[PD] \subseteq S[LC]$ with $L = PK(S)$, where (i) no other IND has the *pointer* P and the referenced relation S, (ii) every *copy* D_i lies outside $PK(R)$ and occurs in no other IND, (iii) every *source* C_i lies outside L and is no copy, and (iv) if the IND has copies, its pointer neither contains $PK(R)$ nor lies inside it.

In particular, every copy follows one source through one pointer, and $\mathcal{K}$ excludes unique keys besides primary keys, copies of copies, and one-to-one replicas, whose pointer is the primary key. Order-sufficiency on $\mathcal{K}$ reduces to a condition on each IND.

Theorem 4.3 (Exactness for $\mathcal{K}$). *Let Σ belong to $\mathcal{K}$. Conditions (1) to (3) are equivalent to (4) every IND $R[PD] \subseteq S[LC]$ in Σ satisfies $C \subseteq Fz$, or both $P \subseteq Fz$ and, for every position i, $C_i \in Fz$ or $D_i \in Fz$. Condition (4) is decidable in linear time.*

That is, every IND must have all its sources frozen, or its pointer frozen and no mutable source with a mutable copy. For FDs and the first two classes, every failure of the condition shows in two entities that must become visible together. On $\mathcal{K}$, the smallest update without a legal order may change four entities, as the following example shows.

Example 4.4 (A four-entity cycle). In Ex. 1.1, the foreign key is implied by the cached-path IND, so the schema belongs to $\mathcal{K}$. Condition (4) holds exactly when *Group.path* is frozen, or when both *Project.gid* and *Project.gpath* are frozen. A workload that renames groups with projects modifies both *path* and *gpath* and violates both alternatives. Freezing only *gpath* violates them as well, although no copy then changes. Indeed, let p_1 move from g_1 to g_2 and p_2 from g_2 to g_1 while the two groups swap their paths /a and /b, so that each project keeps its cached path. In this update, g_2 must become visible no later than p_1, p_1 no later than g_1, g_1 no later than p_2, and p_2 no later than g_2, so the four entities must become visible together.

4.2 Mutable Redundancy

The cycle of Ex. 4.4 needs two groups to exchange their paths, which fresh updates rule out. For fresh updates, class $\mathcal{K}$ separates order-sufficiency from order-insensitivity.

Proposition 4.5 (Fresh updates on class $\mathcal{K}$). *Let Σ belong to $\mathcal{K}$. Every fresh modification update is executable under Fz if and only if every IND $R[PD] \subseteq S[LC]$ in Σ has $C_i \in Fz$ or $D_i \in Fz$ at every position i. Then writing the relations in a topological order of the IND graph, referencing relations first, executes every fresh modification update. Every fresh modification update is order-insensitive under Fz if and only if condition (4) of Thm. 4.3 holds. Otherwise, some fresh update with two changed entities is not executable, or not order-insensitive, respectively.*

Write order matters exactly when the condition for executability holds and the one for order-insensitivity fails, that is, when a mutable pointer leads to a mutable source whose copy is frozen. In Ex. 4.4, freezing only *gpath* thus makes every fresh update executable, some only in the right order. If a group takes a fresh path while its projects move to another group that has the old path, the projects must be written before the group. Props. 3.7 and 4.5 also judge single templates, since the updates of a template are the modification updates under the freeze of every attribute that it does not modify. Applied to each template, they yield our central result.

Theorem 4.6 (Mutable redundancy). *Let a template only modify attributes and have fresh updates. (a) If Σ contains only FDs, the template needs atomic visibility if and only if it modifies the right side A of an elementary FD $X \to A$ in Σ^+ whose left side X is not a superkey. (b) If Σ belongs to $\mathcal{K}$, it needs atomic visibility if and only if it modifies both the source C_i and the copy D_i of an IND $R[PD] \subseteq S[LC]$ in Σ at some position i.*

In both cases the template rewrites a value and its copy, which is the test that our analyzer applies to code (Sec. 6.1). The test stays

valid with unique keys besides the primary key once it counts as frozen every pointer whose frozen copies hold a unique key of the referenced relation (Prop. A.12). It also stays valid with FDs whose right sides no template modifies, as in GitLab (App. A.3). A designer thus sees from the schema and the frozen attributes alone which templates need atomic visibility.

4.3 Conditional INDs

Beyond fresh updates and the exact classes, write order can help even modifications that exchange values. An IND into reference data that no update changes is one example (Ex. A.14), and the one-way conditions on a status that applications maintain are another. A *conditional IND* (CIND) [14] $R[P; f \in A] \subseteq S[L; g \in B]$ requires, for every tuple t of R with $t[f] \in A$, a witness s of t under $R[P] \subseteq S[L]$ with $s[g] \in B$. Here f and g are attributes, and A and B are sets of values. Either condition may be absent. For example, let the groups and projects of Ex. 1.1 carry a flag *arch*. A project that is not archived must belong to a group that is not archived, that is, $Project[gid; arch \in \{0\}] \subseteq Group[gid; arch \in \{0\}]$.

An update *enables* an entity of S if, for some CIND into S, the new tuple of the entity meets the condition on S but its old tuple does not. It *disables* the entity in the converse case. Here an inserted entity has no old tuple, and a deleted entity no new one.

The class C consists of the schemas whose FDs are primary keys and whose INDs are CINDs with $L = PK(S)$ and an acyclic IND graph. Their CINDs into a relation S must all test the same attribute g of S, against sets B that form a chain under inclusion. On this class, enabled and disabled entities decide the write order.

Theorem 4.7 (Conditional INDs). *Let Σ belong to C. Every update, including one that inserts or deletes entities, is executable in three steps: it writes the entities that it enables, referenced relations before referencing ones, then the other changed entities, and last the entities that it disables, referencing relations before referenced ones.*

Hence every schema of C is order-sufficient under every freeze, even for updates that exchange values. Thm. 4.7 extends the classical rule of Prop. 5.4 from insertions and deletions to every update, with enabling in place of insertion and disabling in place of deletion. Indeed, an enabled entity must become visible before the entities that come to need it, and a disabled entity after those that stop needing it. The chain of sets B prevents an update from both enabling and disabling one entity. For the flag *arch*, archiving a group with its projects writes the projects first and the group last.

5 DESIGNING FOR ORDER

Secs. 3 and 4 decide which templates need atomic visibility. Freezing removes the need, shard keys confine it to single shards, and write orders execute the templates that need only order.

5.1 Choosing the Frozen Attributes

To freeze an attribute that the workload modifies, a designer takes its modifications out of the workload. Each of them is either rewritten as *re-keying* in a legal order (Sec. 5.3) or executed with atomic visibility, for example within one shard (Sec. 5.2). Re-keying inserts the entity under a new key with the new value, moves the entities that reference the old entity to it, and deletes the old entity. Both options have a cost, which we weight by the frequency of the templates that modify the attribute. The problem MIN-FREEZE takes a schema Σ, the set W of attributes that the workload modifies, weights on W, and a bound k. It asks whether freezing attributes of W of total weight at most k makes Σ order-sufficient.

With FDs only, Thm. 3.5 yields a unique minimum freeze for every weighting in polynomial time. Besides the primary keys, it contains the attributes on the left side of some elementary FD in Σ^+, and those that such an FD determines when its left side is not a superkey.

INDs make the choice combinatorial. For a schema in $\mathcal{K}$, the *conflict graph* H has the attributes in W as nodes. For every IND $R[PD] \subseteq S[LC]$, it has an edge between every attribute of the pointer P in W and every source of C in W, and an edge between C_i and D_i whenever both are in W. The edges of H are the pairs of attributes that cannot both stay mutable. In Ex. 4.4, H has the edges *gid–path* and *path–gpath*, so its minimal vertex covers are the two alternatives of Thm. 4.3 (Ex. A.15).

Theorem 5.1 (Freezing as vertex cover). *Let Σ belong to $\mathcal{K}$. Freezing a set $Z \subseteq W$ makes Σ order-sufficient if and only if Z is a vertex cover of H. Every graph is the conflict graph of a schema in $\mathcal{K}$, so MIN-FREEZE is NP-hard even on $\mathcal{K}$, by reduction from vertex cover [37].*

The hardness of Thm. 5.1 stems from odd cycles of the undirected IND graph. Indeed, every edge of H joins attributes of two relations that an IND connects. Hence H is bipartite whenever the undirected IND graph is, in particular when the foreign keys form a forest. By Thm. 5.1, our algorithm Freeze (Alg. A.1) only has to cover H. On a bipartite H, it computes a minimum-weight vertex cover as a minimum cut. Otherwise, it computes one of at most twice the minimum weight by the linear-time algorithm of Bar-Yehuda and Even [10]. If only fresh updates must be executable, it drops the edges at pointers (Prop. 4.5). Then H falls apart into stars of a source and its copies, and the cut takes the lighter side of each.

5.2 Choosing Shard Keys

Templates that still need atomic visibility require it only across shards, since an engine writes all entities of one shard atomically. We fix a shard key $SK(R) \subseteq Fz(R)$ for every relation R, so that no entity changes its shard. An update is *executable under SK* if its changed entities can be written in steps that each write entities of one shard atomically, such that the state after every step is legal. Since a hash function may map shard-key values to shards arbitrarily, *hash placement* requires executability under every such map. We call an FD *unstable* if it is not in $\mathcal{G}$. Under FDs, shard keys preserve executability exactly under the following condition.

Theorem 5.2 (Admissible shard keys). *Let Σ contain only FDs, and let $SK(R) \subseteq Fz(R)$ for every relation R. Under hash placement, every modification update is executable under SK if and only if every unstable elementary FD $X \to A$ in Σ^+ satisfies $(X \cap Fz)^+ \supseteq SK$.*

That is, tuples that agree on the frozen attributes of X lie on one shard. Thm. 3.5 is the special case $SK = PK$. In the design of Ex. 3.6, if projects never move, Thm. 5.2 admits only *gid* as the shard key of projects. This shard key turns every rename into a single-shard step (Ex. A.17).

For INDs, entities that must become visible together need one shard, and only co-location puts entities of different relations on one shard. We partition the relations into *co-location groups*, as Citus does [23]: the relations of each have shard keys of one length and share the map from shard-key values to shards. Hash placement then ranges over the maps of all co-location groups. An IND $R[PD] \subseteq S[LC]$ *carries* the shard key if R and S are co-located and Σ implies $t[SK(R)] = s[SK(S)]$ for every tuple t of R and its witness s. The chase decides this, since the IND graph is acyclic.

Theorem 5.3 (Admissible shard keys for $\mathcal{K}$). *Let Σ belong to $\mathcal{K}$, let $SK(R) \subseteq Fz(R)$ for every relation R, and let the relations be partitioned into co-location groups. Under hash placement, every modification update is executable under SK if and only if every IND of Σ that violates condition (4) of Thm. 4.3 carries the shard key.*

That is, a reference that caches a mutable source must store the shard key of the entity that it references if its pointer can change, or if a copy can change together with its source. All other references may cross shards. The equivalence of (1) and (4) in Thm. 4.3 is the special case $SK = PK$ with a co-location group for every relation. The proof of Thm. 5.3 lifts the counterexamples of Thm. 4.3 to every placement by a direct product [27]. For fresh updates, the condition of Thm. 5.3 is sufficient but not always necessary.

5.3 Compiling Write Orders

An executable update still has to be written in a legal order, and the engines differ in how an application can impose one. We first order insertions and deletions, then fix the commit models of the engines, compile an update into the fewest rounds, and last handle failures and slow readers, leaving concurrent updates to App. A.4. In this subsection, an update may also insert and delete entities, which we treat as entities whose old or new version is absent. Updates that only insert or only delete are rarely order-insensitive, since a child written before its parent is an orphan, but they always admit a legal order when the INDs are acyclic.

Proposition 5.4 (Insertions and deletions). *Let the IND graph of Σ be acyclic. An update that only inserts entities is executable by writing the relations in reverse topological order of the IND graph, and an update that only deletes entities is executable by writing them in topological order.*

With a cyclic IND graph, two inserted entities may reference each other, and no order exists.

Updates that mix insertions, deletions and modifications, such as the re-keying of Sec. 5.1, need a graph of their constraints. An update is *binary* if some directed graph $\mathcal{P}$ on Ch, its *precedence graph*, makes an intermediate state $I \oplus T$ legal exactly when no edge of $\mathcal{P}$ leads from an entity outside T to an entity in T. That is, an edge from v to u forbids every state in which u is new and v is old. An update is binary whenever every IND $R[X] \subseteq S[Y]$ in Σ has a key K of S inside Y and no entity takes over the K-value of another, as under foreign keys to primary keys (Prop. A.1).

Commit models. The shards constrain the legal orders of a binary update as follows.

Lemma 5.5 (Commit models). *Let a binary update and an assignment of its changed entities to shards be given. (1) If the update may commit several transactions on one shard, it can be executed by atomic single-shard steps if and only if every strongly connected component of $\mathcal{P}$ lies within one shard. (2) If it commits one transaction on every shard, in an order that the application chooses, it can be executed if and only if merging the entities of every shard into one node, ignoring loops, yields an acyclic graph.*

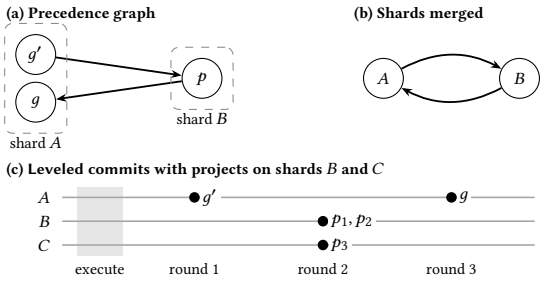


Figure 4: The Template of Ex. 5.6: Precedence, Merged Shards and Leveled Commits

The engines of Sec. 2.1 realize these commit models differently. Single-shard transactions committed in a legal order, which we call ordered commits, realize model (1) on every engine. A single distributed transaction realizes model (2) only in part, since its shards commit in parallel on Citus and after the shard touched first on Vitess. It thus executes an update on Citus only if no edge of $\mathcal{P}$ joins two shards, and on Vitess whenever every such edge leaves the shard touched first. Model (2) is strictly more restrictive than model (1), as the following example shows.

Example 5.6 (Re-keyed rename). In Ex. 1.1, a template inserts a group g', moves a project p from group g to g', and deletes g. It re-keys a rename of g under a freeze of *path*. Each step is legal: g' starts without projects, the move writes the key and the path of g' into the row of p at once, and g is deleted once no project references it. The foreign key gives the precedence chain from g' to p to g of Fig. 4a. If g and g' share a shard A while p lies on a shard B, merging the shards closes the cycle of Fig. 4b. Hence no single distributed transaction executes the template legally, whatever its statement order. Three single-shard transactions, committed in turn, do.

Rounds. Committed one entity at a time, as in Ex. 5.6, ordered commits pay one durable commit per entity, but entities that no path of $\mathcal{P}$ connects may commit in parallel. An execution runs in *rounds* if the single-shard transactions of every round commit in parallel and in any order, and every round starts once the previous one has committed. The precedence graph and the shards fix the number of rounds.

Proposition 5.7 (Minimum rounds). *Let a binary update and an assignment of its changed entities to shards be given, such that every strongly connected component of $\mathcal{P}$ lies within one shard. For an entity e, let $\lambda(e)$ be the largest number of edges across shards on a path of $\mathcal{P}$ that ends in e. Then the update can be executed in d rounds if and only if $d > \lambda(e)$ for every entity e. Round i attains this bound by committing, for every shard, the entities e of that shard with $\lambda(e) = i - 1$ in one transaction.*

Algorithm 1 Leveled Commits

Require: binary update, shards sh of changed entities, precedence graph $\mathcal{P}$, pause δ
Ensure: the update in the fewest rounds, or aborted before any write is visible

1: **for all** components C of $\mathcal{P}$ in topological order **do**
2: $\quad \lambda(C) \leftarrow \max(\{0\} \cup \{\lambda(C') + [sh(C') \neq sh(C)] : C' \rightarrow C\})$
3: $d \leftarrow 1 + \max_C \lambda(C)$
4: **for** $i \leftarrow 1, \ldots, d$ **do**
5: $\quad$ **for all** shards h **do**
6: $\quad\quad$ open $T_{i,h}$ on h for $\bigcup\{C : sh(C) = h,\ \lambda(C) = i - 1\}$
7: execute all writes in parallel, with no-wait locks and an outbox in every $T_{1,h}$
8: **if** a write fails **then**
9: $\quad$ abort every $T_{i,h}$ and **return**
10: **for** $i \leftarrow 1, \ldots, d$ **do**
11: $\quad$ **if** $i > 1$ **then**
12: $\quad\quad$ wait δ
13: $\quad$ commit every $T_{i,h}$ in parallel and await their commits

We call these executions leveled commits. Alg. 1 computes λ on the strongly connected components of $\mathcal{P}$ in linear time (lines 1–2). It groups the entities of every shard and value of λ into one transaction (lines 4–6). The application derives the input $\mathcal{P}$ from its schema. Under foreign keys, an edge leads to a child from the parent that it comes to reference, and from the child to the parent that it stops referencing (Fig. 4a). The number of rounds depends on how often a path of $\mathcal{P}$ changes shards, and not on the numbers of entities or shards. For example, a parent with any number of children on other shards commits in two rounds, and the template of Ex. 5.6 in three, however many projects it moves (Fig. 4c). Every round consists of single-shard transactions, which the application sends in parallel straight to the nodes that hold their shards, whereas 2PC needs a node that coordinates the shards. Leveled commits protect the reader of assumption 5, which reads first what an update writes last. If it sees the new tuple of an entity u, it also sees that of every v with an edge to u, since v commits in an earlier round or together with u and is read after u.

Failures and recovery. Leveled commits also confine the failures of an update. Alg. 1 opens the transactions of all rounds at once and executes their writes in parallel before the first round commits (lines 7–9). A logical failure, such as a violated key, then aborts the update before any write is visible. Only a crash can stop an update between rounds, where it leaves a legal state. After such a crash, a recovery process, the *relay*, commits the remaining rounds from an *outbox*. Every transaction of the first round stores the update and the tuples that it read in this outbox on its shard. The relay validates these tuples, so the update completes or stops in a legal state without overwriting another update (Prop. A.18). An update that must never stop between rounds can instead run as an *ordered 2PC*, which prepares all its transactions before the first round commits, at the price of a log flush per transaction (App. B.2).

A pause between rounds. Readers other than those of assumption 5, such as a join over several shards or a reader that follows a foreign key during a deletion, may see a later round on one shard but miss an earlier one on another. A pause between rounds separates such reads.

Proposition 5.8 (Grace period). *Let leveled commits send the commits of every round at least δ after all transactions of the previous round have acknowledged their commits. Then every reader that reads each shard once, all within less than δ, observes a legal state.*

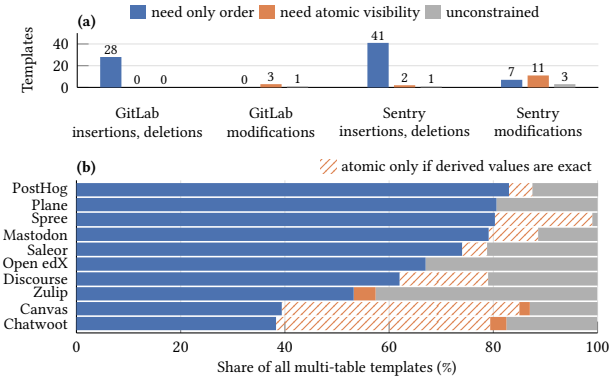


Figure 5: Templates by Requirement: GitLab and Sentry across Databases, by Hand (a), and Ten Further Applications across Tables, by the Analyzer (b)

Indeed, a reader faster than the pause sees the first i rounds and part of round $i + 1$, a state that the proof of Prop. 5.7 shows legal. The pause needs no synchronized clocks, since Prop. 5.8 compares durations, not timestamps, and only assumes that clocks run at the same rate. It adds δ to every round after the first but no message.

6 EXPERIMENTS

Our experiments answer four questions.

(E1) How often does write order suffice in real applications, and how many of their templates do the exact classes cover?
(E2) Do the anomalies that write order prevents occur on Citus and Vitess, and what does write order cost relative to 2PC?
(E3) How fast are the *deciders*, that is, our implementations of the tests of Thms. 3.5 to 5.2?
(E4) How much does the design of Sec. 5 gain over the rules in use today?

Questions (E1) and (E2) test the premise of (Q1) and (Q2): most real updates need write order rather than atomic visibility, and the engines do not supply it. Questions (E3) and (E4) test whether our answers are practical.

6.1 Setup

Applications. We study two applications that already split their data: GitLab [33], whose 1,127 tables lie in three databases, and Sentry [58], whose 323 tables lie in one control database and in cell databases sharded by organization. We classify by hand every template that writes across their databases (App. B.1) as unconstrained, needing only order or needing atomic visibility (Sec. 2.3), from the constraints between its rows (Fig. 5a). A second annotator, who saw neither our labels nor the paper, labeled the templates from the same instructions and agrees with 88% of our labels ($\kappa = 0.71$), and with 94 of the 97 on whether order suffices. *Analyzer.* To test whether GitLab and Sentry are typical, we built a static analyzer, which reads the code of ten further open-source applications [15, 18, 25, 48, 50, 52, 53, 57, 61, 75] on Rails and Django without running it. We chose widely used applications on both frameworks: most keep the data of many tenants, such as accounts,

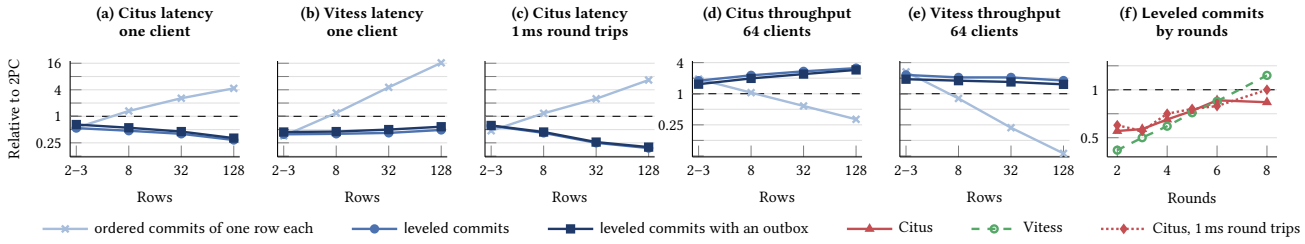


Figure 6: Latency and Throughput of Leveled Commits Relative to the Fastest Two-Phase Commit

realms or shops, and three already split their data across shards, databases or stores. The analyzer collects the foreign keys, the columns that callbacks copy along them and the counters, and groups the writes of every transaction block or method into a template. It labels every template that writes several tables by Prop. 5.4 and Thm. 4.6, assuming that every table may lie on a shard of its own and that modifications write fresh values. Freshness matters only for attributes that an FD or IND references. It holds in GitLab and Sentry, whose copied sources are unique paths, names and tokens, and we assume it for the ten applications.

Engines. We run Citus 14.2 on PostgreSQL 17 with a coordinator and three workers, and Vitess 24 in its 2PC mode on MySQL 8.4 with two shards. In the foreign-key runs, a writer inserts a parent and its child, and a reader that sees the child reads the parent in its next statement, which reverses the order of ordered commits. On Citus, one worker commits with an injected delay, 10 ms unless stated otherwise, which separates the commit points of 2PC. In the copy runs, a writer updates a source and its copy in one transaction while a reader reads both in one statement.

Cost. The cost experiments insert a parent before its children, delete the children before their parent, or move the children to a new parent before deleting the old one, on 2 to 128 rows on alternating shards. Leveled commits send every transaction in one message to the node of its shard. We compare them with ordered commits of one row each and with the fastest of four 2PC variants. These send one statement per row or per shard, to the coordinator or to a worker. The clients write disjoint rows, so that no update waits for another, and in some runs, proxies add 1 ms to every round trip. Both engines commit durably to one storage device that all nodes share, and Vitess also replicates every commit semi-synchronously. Fig. 6 reports median latency and throughput relative to the fastest 2PC, which takes 1.0 to 8.8 ms on Citus and 7.0 to 9.0 ms on Vitess from 2 to 128 rows.

Hardware. All experiments ran on one server with 1 TB of memory and two 64-core AMD EPYC 7742 processors, and the artifact contains the scripts and the full protocol.

6.2 How Often Does Order Suffice?

We first ask whether write order is the remedy that real applications need (E1). Fig. 5a shows that most cross-database write templates need only order in both applications, 28 of 32 in GitLab and 48 of 65 in Sentry. We count each template once, since frequencies are not public, and the share by frequency may be lower. Order fails almost exclusively for modifications. Every template that needs atomic visibility maintains redundant data whose copy or derivation lives on another database, such as the full paths of routes in GitLab and the tokens that Sentry copies into its replica tables. For each of these 16 templates, 3 in GitLab and 13 in Sentry, an audit of the code finds a reader that relies on the copy agreeing with its source. For 13 of the 16, the reader decides authentication or authorization, such as a revoked token that still authenticates. Order still helps some modifications, and Thm. 4.7 explains why. Five of the seven modifications of Sentry that need only order keep one-way conditions, such as an integration that is disabled after its repositories, and the order matters for each of them. The other two also insert or delete rows. Apart from such conditions, a modification that rewrites no copy and moves no pointer relates no rows across databases, so it is unconstrained. The templates that need only order thus insert, delete or move pointers, which the classical rule explains, or keep one-way conditions. In both applications, order is the common case and mutable redundancy (Thm. 4.6) the exception. For the second part of (E1), the exact classes of Sec. 4, extended to unique keys by Prop. A.12, cover 22 of the 32 templates of GitLab and 41 of the 65 of Sentry (App. B.1).

Fig. 5b shows that the analyzer finds the same pattern in the 6,033 multi-table templates of the ten further applications: 99% of the 5,049 constrained ones need only order if derived values may lag behind their rows. Derived values such as counters lie outside our model, since they are neither FDs nor INDs. If they must stay exact, the share drops to 78%. The share of 99% is too high, since the analyzer misses some copies. Blind hand labels of 135 templates from the ten applications, made without the analyzer's verdicts, put it at about 82% (95% interval 74–90%) if derived values may lag. The sample is stratified by application and analyzer verdict, and every stratum is weighted by its size. The hand labels trace every template that needs atomic visibility to a copy or a derived value. Against these labels and ours, the analyzer is almost always right when it reports atomic visibility (Table 1 in App. B).

However, write order protects only the readers of assumption 5. An audit of the read paths of the 76 templates that need only order shows that most readers read in the protected direction or tolerate a missing parent. For 47 of these templates, write order alone protects every reader, since each reverses the write order or reads only one of the related rows. Each of the other 29 has a reader that reads in the write order, mostly one that loads the parent of a child during a deletion. Both applications mostly expect such parents to be missing, since their own order leaves dangling children to a background task. Sentry tolerates a missing parent at 80 of the 88 places where

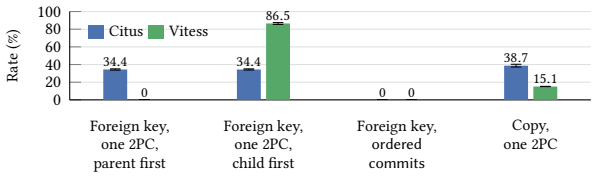


Figure 7: Orphans per Run and Inconsistent Copies per Read, with 95% Confidence Intervals and a 10 ms Commit Delay on One Citus Worker in the Foreign-Key Runs

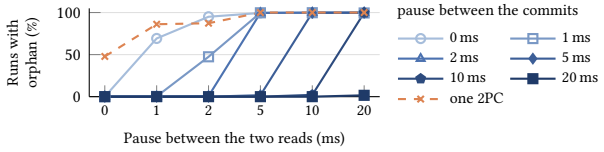


Figure 8: Orphans during Deletions on Citus without Injected Delay, by Pause between the Reads and between the Commits

its code follows such a reference. For 9 templates such a reader raises an error, as in the incident of Sec. 1, and we reported them to both projects. A pause between rounds protects such readers if they are fast (Sec. 6.3).

6.3 Does Write Order Matter on Real Engines?

Having seen that most templates need only order, we ask whether 2PC provides it, and what leveled commits cost (E2). Fig. 7 shows that one 2PC exposes the anomalies that write order prevents, whereas ordered commits prevent them on both engines. On Vitess, where the shard touched first commits before the others (Sec. 2.1), a reader saw an orphan in 86% of the runs that wrote the child first and in none of those that wrote the parent first. On Citus, the commit points of the workers lie so close together that only 0.1% of the runs exposed an orphan without delay. Once one worker commits 1 ms later or more, a reader saw an orphan in almost every run whose parent lay on that worker, whatever the statement order. As the parent lay on that worker in a third of the runs, one 2PC exposed an orphan in 34% of the runs from 1 ms up to 10 ms. Hence a parent and its child need separate transactions on Citus but only the statement order on Vitess (Lem. 5.5). For the templates that need atomic visibility, 2PC is no remedy either. A transaction that wrote a source and its copy exposed an inconsistent pair in 15% of reads on Vitess and 39% on Citus.

Figs. 6a to 6c show that leveled commits also take less time than the fastest 2PC at every number of rows, with and without the 1 ms proxies. At 128 rows they take less than a third of its time on Citus and half of it on Vitess, and a fifth behind the 1 ms proxies on Citus. Most of this advantage stems from the access path that single-shard transactions allow (Sec. 5.3, App. B.2). Indeed, leveled commits send the statements of every shard in one message to the node that holds it, whereas 2PC routes every statement through the coordinator, even when it enters at a worker.

The cost of write order lies in its rounds and not in its rows. Ordered commits of one row each take four times as long as 2PC on Citus and sixteen times on Vitess at 128 rows, since they pay one durable commit per row. Leveled commits instead need the fewest rounds (Prop. 5.7). The number of rounds, in turn, bounds the advantage of leveled commits. Fig. 6f shows that the advantage shrinks as rounds are added, for chains of 2 to 8 rows that lie on different nodes and need one round per row. It holds up to eight rounds on Citus, but only up to six on Vitess and behind 1 ms proxies, so six rounds is the common bound. Indeed, every round adds a round trip and a durable commit, whereas 2PC pays its prepare and commit rounds once.

The advantage carries over from latency to throughput: with 64 clients, leveled commits sustain 1.8 to 3.1 times the throughput of 2PC at every size on both engines (Figs. 6d and 6e).

Recovery and locks, which prevent the orphans of failures and conflicts, keep most of this advantage (App. B.2). An outbox raises the latency of leveled commits by at most a fifth. When a proxy dropped commit messages, leveled commits with an outbox left no orphan, whereas one 2PC left orphans until the periodic recovery of Citus ran. Locks on shared parents prevent the orphans of conflicting updates and cost an additional 0.14 ms. Leveled commits sent straight to the shards thus protect the readers of assumption 5 in less time than one 2PC, and protect them even when commit messages are lost or updates conflict.

Assumption 5 matters in practice, since read direction decides whether write order alone protects a reader. Fig. 8 shows this for deletions on Citus without injected delay: a reader that follows the foreign key from the child saw an orphan under ordered commits in 69% of the runs once 1 ms separated its two reads. A reader that lists the children of the parent saw none in 6,000 runs. Indeed, the reader from the child sees an orphan whenever its two reads enclose both commits, which no order prevents, and insertions swap the roles of the two readers. One 2PC, the reference line, exposes the reader from the child more often, in 48% of the runs even without a pause, since its two commits fall more easily between the two reads.

A pause between the two commits closes this window for every reader that reads faster than the pause, as Prop. 5.8 predicts. In Fig. 8, orphans appear only once the pause between the reads reaches the pause between the commits. Among 700,952 pairs of reads that took no longer than the pause, in deletions and insertions alike, none saw an orphan. Write order with a pause protects every reader that finishes within the pause, and only slower readers must tolerate missing parents (Sec. 6.2).

6.4 How Fast Is Design, and What Does It Gain?

Having seen that write order matters on real engines, we ask whether a design tool can decide order-sufficiency fast (E3), and what the design of Sec. 5 gains over the rules in use today (E4). The deciders are fast: every decider answers within 4 s on random schemas with 10,000 relations and within 0.4 s on the 1,127 relations of GitLab, and a search for counterexamples confirms every verdict. The minimum cut finds the optimal freeze over about 70,000 attributes in 0.6 s (App. B.3).

For (E4), Thm. 5.3 explains why GitLab can shard by its declared keys without atomic visibility. GitLab maintains 903 copies in its schema or code, 808 of sharding keys (its term for shard keys) and 95 of other attributes that callbacks write. Once the sharding keys are frozen, none of the 808 violates condition (4) of Thm. 4.3, and

co-location on the declared keys carries the 4 of the 95 that do (Fig. 9 in App. B). GitLab modifies sharding keys only when it transfers, imports or backfills data. Counted as modifications, these writes raise the violations to 45, of which co-location carries only 35. Only the templates that transfer data then need re-keying or atomic visibility.

The design also gains over the rules in use today. The rule of Citus [20] puts the shard key inside every unique key, so that every shard checks uniqueness on its own data. It admits a single-column shard key for 527 of the 1,127 tables of GitLab. Thm. 5.2 admits one for 1,040 tables, whose unique keys determine the shard key through copied columns, so that a shard still checks them locally [35]. The simplest rule, which freezes every copied source, overpays as well. Under skewed update frequencies, the optimal freeze saves 36 to 70% of its weight (Fig. 10 in App. B). Exact conditions thus admit shard keys and freezes that these rules forbid or overpay for.

7 RELATED WORK

Coordination avoidance. Invariant confluence [3] tells whether an operation can run without coordination. Indigo [9], IPA [8] and the proof rule of Gotsman et al. [34] decide which operations of a replicated database must synchronize to preserve invariants, or repair them so that they need not. RedBlue consistency [42] and the homeostasis protocol [56] let the operations that preserve the invariants run without coordination, and Salt [68] and Callas [69] confine coordination to the transactions that need it. Robustness analyses decide whether transaction templates run serializably under snapshot isolation [11, 16, 28] or read committed [38, 65]. These analyses treat the writes of one operation or transaction as atomic, whereas we split the constraints that need coordination into those that need only order and those that need atomic visibility.

Write ordering. Like soft updates [29], object-relational mappers and the cascades of Rails (`dependent: :destroy`) and Django (`on_delete`) order writes along a fixed dependency rule, but within one transaction, whose statement order protects no reader on Citus (Lem. 5.5). Sagas [31] commit the steps of a long transaction separately and compensate on failure, and the transactional outbox [55] makes such steps reliable, but both leave the consistency of the intermediate states to the application. Semantic consistency [30], multilevel atomicity [46], commitment ordering [54] and quasi serializability [26] interleave or order whole transactions, at declared breakpoints or by their commits, whereas we order the writes within one update. Transaction chopping [59] and chains [72] also commit the pieces of a transaction in turn, but derive the pieces from a static analysis of all transactions. Leveled commits instead derive the order from the schema and promise legal states to readers that no analysis sees. Markowitz [47] showed when the order of referential actions does not change their effect, whereas we ask which orders keep every intermediate state legal.

Schema design. Normalization targets redundancy, with normal forms such as the elementary key normal form [71], the object normal form of Biskup [13], and normal forms parameterized by the numbers of minimal keys and FDs [73, 74]. LeDoux and Parker [40] related BCNF to the anomalies of given updates. Levene and Vincent [41] justified the inclusion dependency normal form, whose INDs are acyclic and reference keys as in our class $\mathcal{A}$, by the absence of redundancy and update anomalies. Bravo et al. [14] introduced conditional INDs for data cleaning and studied their implication. These works consider a database whose updates are atomic, whereas we ask which constraints survive the intermediate states of updates on sharded engines, relative to the frozen attributes of a workload. For fresh updates, our answer is again redundancy, but only redundancy among mutable attributes.

Sharded engines and practice. Helland [36] advocated designs without distributed transactions, which leave the constraints across entities to the application. In practice, applications enforce many integrity constraints themselves [4], maintain constraints across microservices by hand [39] and coordinate operations with ad hoc transactions [63]. Our tests tell such applications where write order suffices. Schism [24] and Horticulture [51] partition a database so that few transactions of a workload span partitions, whereas Thm. 5.2 asks which shard keys let the remaining updates do without atomic visibility. F1 [60] interleaves child rows under their root and PREF [70] co-locates rows along foreign keys, which Thm. 5.3 requires only for the INDs that need atomic visibility.

None of these works characterizes when a legal write order exists, or how schema and placement decide it.

8 CONCLUSION

We have studied which updates on sharded SQL engines need atomic visibility and when write order alone keeps the constraints of a schema satisfied. For (Q1), under FDs and foreign keys with cached columns, a fresh update needs atomic visibility exactly when it rewrites a value and its copy. For arbitrary modifications under FDs, the update-stable consequences recover the object normal form of Biskup. For (Q2), the frozen attributes decide the shard keys that confine atomic visibility to one shard. The other updates compile to leveled commits, which need the fewest rounds and take less time than 2PC for up to six rounds. In GitLab, Sentry and ten further applications, most write templates across databases or tables need only order.

Future work will model derived values such as counters, which separate the two shares of Sec. 6.2, to tell which of them may lag behind their rows. The theory leaves open FDs together with class $\mathcal{K}$ and the acyclic and conditional INDs outside our classes (App. A.3). Atomic visibility is the price of mutable redundancy, and write order is a design decision that the schema can justify.

REFERENCES

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[2] Peter Bailis, Aaron Davidson, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. Highly Available Transactions: Virtues and Limitations. *Proceedings of the VLDB Endowment* 7, 3 (2013), 181–192. https://doi.org/10.14778/2732232.2732237

[3] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2014. Coordination Avoidance in Database Systems. *Proceedings of the VLDB Endowment* 8, 3 (2014), 185–196. https://doi.org/10.14778/2735508.2735509

[4] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2015. Feral Concurrency Control: An Empirical Investigation of Modern Application Integrity. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1327–1342. https://doi.org/10.1145/2723372.2737784

[5] Peter Bailis, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2016. Scalable Atomic Visibility with RAMP Transactions. *ACM Transactions on Database Systems* 41, 3 (2016), 15:1–15:45. https://doi.org/10.1145/2909870

[6] Peter Bailis, Alan Fekete, Joseph M. Hellerstein, Ali Ghodsi, and Ion Stoica. 2014. Scalable Atomic Visibility with RAMP Transactions. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. 27–38. https://doi.org/10.1145/2588555.2588562

[7] Peter Bailis, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. Bolt-on Causal Consistency. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 761–772. https://doi.org/10.1145/2463676.2465279

[8] Valter Balegas, Sérgio Duarte, Carla Ferreira, Rodrigo Rodrigues, and Nuno M. Preguiça. 2018. IPA: Invariant-Preserving Applications for Weakly Consistent Replicated Databases. *Proceedings of the VLDB Endowment* 12, 4 (2018), 404–418. https://doi.org/10.14778/3297753.3297760

[9] Valter Balegas, Sérgio Duarte, Carla Ferreira, Rodrigo Rodrigues, Nuno M. Preguiça, Mahsa Najafzadeh, and Marc Shapiro. 2015. Putting Consistency Back into Eventual Consistency. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*. 6:1–6:16. https://doi.org/10.1145/2741948.2741972

[10] Reuven Bar-Yehuda and Shimon Even. 1981. A Linear-Time Approximation Algorithm for the Weighted Vertex Cover Problem. *Journal of Algorithms* 2, 2 (1981), 198–203. https://doi.org/10.1016/0196-6774(81)90020-1

[11] Sidi Mohamed Beillahi, Ahmed Bouajjani, and Constantin Enea. 2019. Checking Robustness Against Snapshot Isolation. In *Computer Aided Verification (CAV) (Lecture Notes in Computer Science)*, Vol. 11562. Springer, 286–304. https://doi.org/10.1007/978-3-030-25543-5_17

[12] Carsten Binnig, Stefan Hildenbrand, Franz Färber, Donald Kossmann, Juchang Lee, and Norman May. 2014. Distributed Snapshot Isolation: Global Transactions Pay Globally, Local Transactions Pay Locally. *The VLDB Journal* 23, 6 (2014), 987–1011. https://doi.org/10.1007/s00778-014-0359-9

[13] Joachim Biskup. 1989. Boyce-Codd Normal Form and Object Normal Forms. *Inform. Process. Lett.* 32, 1 (1989), 29–33. https://doi.org/10.1016/0020-0190(89)90065-3

[14] Loreto Bravo, Wenfei Fan, and Shuai Ma. 2007. Extending Dependencies with Conditions. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*. 243–254. https://www.vldb.org/conf/2007/papers/research/p243-bravo.pdf

[15] Canvas LMS 2026. Canvas LMS: Source Code Repository. https://github.com/instructure/canvas-lms. Commit 1c9f0bb8, accessed October 2026.

[16] Andrea Cerone and Alexey Gotsman. 2018. Analysing Snapshot Isolation. *Journal of the ACM* 65, 2 (2018), 11:1–11:41. https://doi.org/10.1145/3152396

[17] Ashok K. Chandra and Moshe Y. Vardi. 1985. The Implication Problem for Functional and Inclusion Dependencies is Undecidable. *SIAM J. Comput.* 14, 3 (1985), 671–677. https://doi.org/10.1137/0214049

[18] Chatwoot 2026. Chatwoot: Source Code Repository. https://github.com/chatwoot/chatwoot. Commit 61c9ef44, accessed October 2026.

[19] Audrey Cheng, Xiao Shi, Lu Pan, Anthony Simpson, Neil Wheaton, Shilpa Lawande, Nathan Bronson, Peter Bailis, Natacha Crooks, and Ion Stoica. 2021. RAMP-TAO: Layering Atomic Transactions on Facebook's Online TAO Data Store. *Proceedings of the VLDB Endowment* 14, 12 (2021), 3014–3027. https://doi.org/10.14778/3476311.3476379

[20] Citus 2026. Citus Documentation: Creating and Modifying Distributed Objects (DDL). https://docs.citusdata.com/en/stable/develop/reference_ddl.html. Accessed October 2026.

[21] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. 2013. Spanner: Google's Globally Distributed Database. *ACM Transactions on Computer Systems* 31, 3 (2013), 8:1–8:22. https://doi.org/10.1145/2491245

[22] Stavros S. Cosmadakis, Paris C. Kanellakis, and Moshe Y. Vardi. 1990. Polynomial-Time Implication Problems for Unary Inclusion Dependencies. *Journal of the ACM* 37, 1 (1990), 15–46. https://doi.org/10.1145/78935.78937

[23] Umur Cubukcu, Ozgun Erdogan, Sumedh Pathak, Sudhakar Sannakkayala, and Marco Slot. 2021. Citus: Distributed PostgreSQL for Data-Intensive Applications. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2490–2502. https://doi.org/10.1145/3448016.3457551

[24] Carlo Curino, Evan P. C. Jones, Yang Zhang, and Samuel Madden. 2010. Schism: A Workload-Driven Approach to Database Replication and Partitioning. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 48–57. https://doi.org/10.14778/1920841.1920853

[25] Discourse 2026. Discourse: Source Code Repository. https://github.com/discourse/discourse. Commit 630b787e, accessed October 2026.

[26] Weimin Du and Ahmed K. Elmagarmid. 1989. Quasi Serializability: A Correctness Criterion for Global Concurrency Control in InterBase. In *Proceedings of the 15th International Conference on Very Large Data Bases (VLDB)*. 347–355.

[27] Ronald Fagin. 1982. Horn Clauses and Database Dependencies. *Journal of the ACM* 29, 4 (1982), 952–985. https://doi.org/10.1145/322344.322347

[28] Alan Fekete, Dimitrios Liarokapis, Elizabeth O'Neil, Patrick O'Neil, and Dennis Shasha. 2005. Making Snapshot Isolation Serializable. *ACM Transactions on Database Systems* 30, 2 (2005), 492–528. https://doi.org/10.1145/1071610.1071615

[29] Gregory R. Ganger, Marshall Kirk McKusick, Craig A. N. Soules, and Yale N. Patt. 2000. Soft Updates: A Solution to the Metadata Update Problem in File Systems. *ACM Transactions on Computer Systems* 18, 2 (2000), 127–153. https://doi.org/10.1145/350853.350863

[30] Hector Garcia-Molina. 1983. Using Semantic Knowledge for Transaction Processing in a Distributed Database. *ACM Transactions on Database Systems* 8, 2 (1983), 186–213. https://doi.org/10.1145/319983.319985

[31] Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*. 249–259. https://doi.org/10.1145/38713.38742

[32] GitLab 2026. Delete Orphaned States in Helm Metadata Cache Backfill. https://gitlab.com/gitlab-org/gitlab/-/merge_requests/246021. Merge request 246021, fixing the severity-1 upgrade failure gitlab-org/gitlab#605940, accessed October 2026.

[33] GitLab 2026. GitLab: Source Code Repository. https://gitlab.com/gitlab-org/gitlab. Commit 9d4cbde6, accessed October 2026.

[34] Alexey Gotsman, Hongseok Yang, Carla Ferreira, Mahsa Najafzadeh, and Marc Shapiro. 2016. 'Cause I'm Strong Enough: Reasoning About Consistency Choices in Distributed Systems. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 371–384. https://doi.org/10.1145/2837614.2837625

[35] Ashish Gupta and Jennifer Widom. 1993. Local Verification of Global Integrity Constraints in Distributed Databases. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. 49–58. https://doi.org/10.1145/170035.170048

[36] Pat Helland. 2007. Life beyond Distributed Transactions: An Apostate's Opinion. In *Third Biennial Conference on Innovative Data Systems Research (CIDR)*. 132–141.

[37] Richard M. Karp. 1972. Reducibility among Combinatorial Problems. In *Complexity of Computer Computations*. Springer, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9

[38] Bas Ketsman, Christoph Koch, Frank Neven, and Brecht Vandevoort. 2020. Deciding Robustness for Lower SQL Isolation Levels. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. 315–330. https://doi.org/10.1145/3375395.3387655

[39] Rodrigo Laigner, Yongluan Zhou, Marcos Antonio Vaz Salles, Yijian Liu, and Marcos Kalinowski. 2021. Data Management in Microservices: State of the Practice, Challenges, and Research Directions. *Proceedings of the VLDB Endowment* 14, 13 (2021), 3348–3361. https://doi.org/10.14778/3484224.3484232

[40] Carol Helfgott LeDoux and D. Stott Parker. 1982. Reflections on Boyce-Codd Normal Form. In *Proceedings of the 8th International Conference on Very Large Data Bases (VLDB)*. 131–141.

[41] Mark Levene and Millist W. Vincent. 2000. Justification for Inclusion Dependency Normal Form. *IEEE Transactions on Knowledge and Data Engineering* 12, 2 (2000), 281–291. https://doi.org/10.1109/69.842267

[42] Cheng Li, Daniel Porto, Allen Clement, Johannes Gehrke, Nuno M. Preguiça, and Rodrigo Rodrigues. 2012. Making Geo-Replicated Systems Fast as Possible, Consistent when Necessary. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 265–278. https://www.usenix.org/conference/osdi12/technical-sessions/presentation/li

[43] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. 2011. Don't Settle for Eventual: Scalable Causal Consistency for Wide-Area Storage with COPS. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*. 401–416. https://doi.org/10.1145/2043556.2043593

[44] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. 2013. Stronger Semantics for Low-Latency Geo-Replicated Storage. In *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 313–328. https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/lloyd

[45] Haonan Lu, Siddhartha Sen, and Wyatt Lloyd. 2020. Performance-Optimal Read-Only Transactions. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 333–349. https://www.usenix.org/conference/osdi20/presentation/lu

[46] Nancy A. Lynch. 1983. Multilevel Atomicity: A New Correctness Criterion for Database Concurrency Control. *ACM Transactions on Database Systems* 8, 4 (1983), 484–502. https://doi.org/10.1145/319996.319999

[47] Victor M. Markowitz. 1991. Safe Referential Integrity Structures in Relational Databases. In *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB)*. 123–132.

[48] Mastodon 2026. Mastodon: Source Code Repository. https://github.com/mastodon/mastodon. Commit 7f90e0fd, accessed October 2026.

[49] John C. Mitchell. 1983. The Implication Problem for Functional and Inclusion Dependencies. *Information and Control* 56, 3 (1983), 154–173. https://doi.org/10.1016/S0019-9958(83)80002-3

[50] Open edX Platform 2026. Open edX Platform: Source Code Repository. https://github.com/openedx/openedx-platform. Commit d7859be1, accessed October 2026.

[51] Andrew Pavlo, Carlo Curino, and Stanley B. Zdonik. 2012. Skew-Aware Automatic Database Partitioning in Shared-Nothing, Parallel OLTP Systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 61–72. https://doi.org/10.1145/2213836.2213844

[52] Plane 2026. Plane: Source Code Repository. https://github.com/makeplane/plane. Commit 4345fb3e, accessed October 2026.

[53] PostHog 2026. PostHog: Source Code Repository. https://github.com/PostHog/posthog. Commit 069e3b3a, accessed October 2026.

[54] Yoav Raz. 1992. The Principle of Commitment Ordering, or Guaranteeing Serializability in a Heterogeneous Environment of Multiple Autonomous Resource Managers Using Atomic Commitment. In *Proceedings of the 18th International Conference on Very Large Data Bases (VLDB)*. 292–312.

[55] Chris Richardson. 2018. *Microservices Patterns: With Examples in Java*. Manning Publications.

[56] Sudip Roy, Lucja Kot, Gabriel Bender, Bailu Ding, Hossein Hojjat, Christoph Koch, Nate Foster, and Johannes Gehrke. 2015. The Homeostasis Protocol: Avoiding Transaction Coordination Through Program Analysis. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1311–1326. https://doi.org/10.1145/2723372.2723720

[57] Saleor 2026. Saleor: Source Code Repository. https://github.com/saleor/saleor. Commit 0c0e2697, accessed October 2026.

[58] Sentry 2026. Sentry: Source Code Repository. https://github.com/getsentry/sentry. Commit 13ccaecc, accessed October 2026.

[59] Dennis E. Shasha, François Llirbat, Eric Simon, and Patrick Valduriez. 1995. Transaction Chopping: Algorithms and Performance Studies. *ACM Transactions on Database Systems* 20, 3 (1995), 325–363. https://doi.org/10.1145/211414.211427

[60] Jeff Shute, Radek Vingralek, Bart Samwel, Ben Handy, Chad Whipkey, Eric Rollins, Mircea Oancea, Kyle Littlefield, David Menestrina, Stephan Ellner, John Cieslewicz, Ian Rae, Traian Stancescu, and Himani Apte. 2013. F1: A Distributed SQL Database That Scales. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1068–1079. https://doi.org/10.14778/2536222.2536232

[61] Spree Commerce 2026. Spree Commerce: Source Code Repository. https://github.com/spree/spree. Commit c6a498f7, accessed October 2026.

[62] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. 2020. CockroachDB: The Resilient Geo-Distributed SQL Database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1493–1509. https://doi.org/10.1145/3318464.3386134

[63] Chuzhe Tang, Zhaoguo Wang, Xiaodong Zhang, Qianmian Yu, Binyu Zang, Haibing Guan, and Haibo Chen. 2022. Ad Hoc Transactions in Web Applications: The Good, the Bad, and the Ugly. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. 4–18. https://doi.org/10.1145/3514221.3526120

[64] Alejandro Z. Tomsic, Manuel Bravo, and Marc Shapiro. 2018. Distributed Transactional Reads: The Strong, the Quick, the Fresh & the Impossible. In *Proceedings of the 19th International Middleware Conference (Middleware)*. 120–133. https://doi.org/10.1145/3274808.3274818

[65] Brecht Vandevoort, Bas Ketsman, Christoph Koch, and Frank Neven. 2021. Robustness against Read Committed for Transaction Templates. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2141–2153. https://doi.org/10.14778/3476249.3476268

[66] Vitess 2026. Vitess: A Database Clustering System for Horizontal Scaling of MySQL. https://vitess.io. Accessed October 2026.

[67] Vitess 2026. Vitess Documentation: Distributed Transactions. https://vitess.io/docs/24.0/reference/features/distributed-transaction/. Version 24.0, accessed October 2026.

[68] Chao Xie, Chunzhi Su, Manos Kapritsos, Yang Wang, Navid Yaghmazadeh, Lorenzo Alvisi, and Prince Mahajan. 2014. Salt: Combining ACID and BASE in a Distributed Database. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 495–509. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/xie

[69] Chao Xie, Chunzhi Su, Cody Littley, Lorenzo Alvisi, Manos Kapritsos, and Yang Wang. 2015. High-Performance ACID via Modular Concurrency Control. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP)*. 279–294. https://doi.org/10.1145/2815400.2815430

[70] Erfan Zamanian, Carsten Binnig, and Abdallah Salama. 2015. Locality-Aware Partitioning in Parallel Database Systems. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 17–30. https://doi.org/10.1145/2723372.2723718

[71] Carlo Zaniolo. 1982. A New Normal Form for the Design of Relational Database Schemata. *ACM Transactions on Database Systems* 7, 3 (1982), 489–499. https://doi.org/10.1145/319732.319749

[72] Yang Zhang, Russell Power, Siyuan Zhou, Yair Sovran, Marcos K. Aguilera, and Jinyang Li. 2013. Transaction Chains: Achieving Serializability with Low Latency in Geo-Distributed Storage Systems. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. 276–291. https://doi.org/10.1145/2517349.2522729

[73] Zhuoxing Zhang, Wu Chen, and Sebastian Link. 2023. Composite Object Normal Forms: Parameterizing Boyce-Codd Normal Form by the Number of Minimal Keys. *Proceedings of the ACM on Management of Data* 1, 1, Article 13 (2023), 25 pages. https://doi.org/10.1145/3588693

[74] Zhuoxing Zhang and Sebastian Link. 2025. Synthesizing Third Normal Form Schemata that Minimize Integrity Maintenance and Update Overheads: Parameterizing 3NF by the Numbers of Minimal Keys and Functional Dependencies. *Proceedings of the ACM on Management of Data* 3, 3, Article 225 (2025), 25 pages. https://doi.org/10.1145/3725362

[75] Zulip 2026. Zulip: Source Code Repository. https://github.com/zulip/zulip. Commit 68812981, accessed October 2026.

A PROOFS

This appendix proves the results of Secs. 3 to 5 in the order in which they appear, together with the examples and further results that the body defers to it. Sec. A.1 first treats the binary updates of Sec. 5.3, since the running example and several later proofs read their precedence graphs. Throughout, instances are finite, and implication and the closure Σ^+ are taken in the finite sense. For an instance I and an entity e, we write $I(e)$ for the tuple of e in I.

A.1 Binary Updates

Prop. A.1 gives a sufficient condition for an update to be binary and reads off its precedence graph. An inserted entity has an absent version in I, and a deleted one in J. An absent version violates no FD, needs no witness and witnesses nothing.

Proposition A.1 (Binary updates). *Let every IND $R[X] \subseteq S[Y]$ in Σ have a key K of S inside Y, and let no entity agree on K in J with another entity in I. Then the update is binary, and its precedence graph has an edge from v to u exactly when (i) the new tuple of u and the old tuple of v violate an FD in Σ, (ii) the new tuple of u has a witness under some IND only in the new tuple of v, or (iii) the old tuple of v has a witness under some IND only in the old tuple of u.*

Proof. An intermediate state shows every changed entity in its old or its new version, and every other entity in its only version. An FD relates two tuples, and two versions that violate it are not both old, since I is legal, nor both new, since J is legal, nor do they involve an unchanged entity, since its version occurs in I and in J. Hence the state satisfies the FDs exactly when it shows no pair of case (i). Consider an IND $R[X] \subseteq S[Y]$, a version t of an entity e, and two versions s and s' of entities w and w' with $s[Y] = s'[Y] = t[X]$. Then s and s' agree on K, so $w = w'$ by the hypothesis if one of them is old and the other new, and by the key K of I or J otherwise. Thus all candidate witnesses of t are versions of one entity w. If t is new, some version of w is a witness, since J is legal, and the state fails the IND at t only if it shows the old version of w while only the new one is a witness, which is case (ii) with $u = e$ and $v = w$, as w then changes. Symmetrically, if t is old, the state fails the IND at t only in case (iii) with $v = e$ and $u = w$. Hence a state is legal exactly when it shows no forbidden pair, that is, when no edge leads into the set of entities that it shows new from outside. □

The hypothesis is needed. If an update deletes an entity w' of S and inserts an entity w with the same value of K, while a changed entity e of R starts to reference this value, then the state that shows e new and w' deleted is illegal, although the states with only one of them are legal, and no precedence graph expresses this. For modification updates, the hypothesis holds whenever $K \subseteq Fz(S)$ or the update is fresh (Sec. 3.3), and inserted entities meet it when they take values of K that I does not contain.

On the running example, Prop. A.1 reads as follows.

Example A.2 (Moves and renames). Consider again Ex. 1.1. Moving a project changes a single entity, so every move is executable. Renaming a group g with a project p is not executable, since the old g is no witness of the new p for the cached-path IND, nor the new g of the old p. By Prop. A.1, the update is binary, with precedence edges from g to p and back. Hence g and p must become visible together, and the schema is not order-sufficient for this workload.

A.2 Proofs for Sec. 3

The running example illustrates the update-stable consequences of Sec. 3.1.

Example A.3 (Unstable cached paths). In Ex. 1.1, $M^* = Fz$ because *Project.gid* is not frozen. The foreign key belongs to $\mathcal{G}$ because its right side *gid* is frozen. The cached-path IND does not, because its right side contains the mutable attribute *path* and its left side is mutable. Hence $\mathcal{G}$ misses exactly the constraint that a rename violates. If *Project.gid* and *Group.path* were frozen, M^* would also contain *Project.gpath*, since it copies the frozen path of the group that the frozen pointer *gid* identifies.

We start with the invariant of the frozen closure.

Proof of Lem. 3.2. Let (I, J) be a modification update. We show by induction on the construction of M^* as a least fixpoint that $I(e)$ and $J(e)$ agree on $M^*(R)$ for every entity e of every relation R. The base case Fz holds by Def. 2.1. For the inductive step, let the rule add X_i to $M^*(R)$ through an IND $R[X] \subseteq S[Y]$ in Σ^+, where Y contains a key K of S with $K \subseteq M^*(S)$ and the attributes X_K of X at the positions of K lie in $M^*(R)$. Let e be an entity of R, and let $u = I(e)$ and $u' = J(e)$. Since I and J satisfy the IND, there are entities w and w' of S with $I(w)[Y] = u[X]$ and $J(w')[Y] = u'[X]$. By the induction hypothesis, $u[X_K] = u'[X_K]$ and $I(w)[K] = J(w)[K]$. Then $J(w)[K] = u[X_K] = u'[X_K] = J(w')[K]$, and $w = w'$ because J satisfies the key K. Consequently, $u'[X_i] = J(w)[Y_i] = I(w)[Y_i] = u[X_i]$ whenever $Y_i \in M^*(S)$. □

The invariant yields the claim made after Def. 3.3.

Lemma A.4. *Every constraint in $\mathcal{G}$ holds in every intermediate state of every modification update.*

Proof. Let (I, J) be a modification update, and let $L = I \oplus T$ for some $T \subseteq Ch$. Every tuple of L is $I(e)$ or $J(e)$ for an entity e, and $I(e)$ and $J(e)$ agree on M^* by Lem. 3.2. Consider first an FD $X \to A$ in $\mathcal{G}$, and let the tuples of two distinct entities e and f agree on X in L. If X contains a key $K \subseteq M^*$, then $I(e)$ and $I(f)$ agree on K, which contradicts the key K in I. If $X \cup \{A\} \subseteq M^*$, then $I(e)$ and $I(f)$ carry the values of the two tuples on $X \cup \{A\}$, so the two tuples agree on A because I satisfies the FD. Consider next an IND $R[X] \subseteq S[Y]$ in $\mathcal{G}$, and let t be the tuple of an entity e of R in L. If $Y \subseteq M^*(S)$, then I, J and L have the same projection on $S[Y]$, and $t[X]$ lies in the projection of I or of J. Otherwise, $X \subseteq M^*(R)$, and Y contains a key K of S with $K \subseteq M^*(S)$. Then $t[X] = I(e)[X] = J(e)[X]$, and there are entities w and w' of S with $I(w)[Y] = t[X] = J(w')[Y]$. By Lem. 3.2, $J(w)[K] = I(w)[K] = J(w')[K]$, so $w = w'$. The tuple of w in L is a witness for t. □

Proof of Thm. 3.4. Every intermediate state is a finite instance that satisfies $\mathcal{G}$ by Lem. A.4. Since $\mathcal{G}$ implies Σ, the state satisfies Σ. Hence every modification update is order-insensitive, and in particular executable. □

For FDs we use a classical fact: an instance of a relation with two tuples satisfies a set of FDs if and only if the attributes on which

the tuples agree, their *agree set*, form a closed set. For a relation R with attributes U, we write $N = U \setminus Fz(R)$ for its mutable attributes. Two tuples t_1 and t_2 *swap their mutable parts* if they are replaced by t_1', which agrees with t_1 on Fz and with t_2 on N, and by t_2', defined symmetrically. The agree set of t_1' and t_2' is that of t_1 and t_2, and the agree set of t_1' and t_2 is the union of N and the frozen part of the agree set of t_1 and t_2. The following lemma shows that condition (4) of Thm. 3.5 freezes every minimal key, which the remark on BCNF in Sec. 3.3 uses and the proof of Thm. 4.1 needs.

Lemma A.5. *Let Σ contain only FDs and satisfy condition (4) of Thm. 3.5. Then every minimal key of every relation is frozen.*

Proof. Assume that a minimal key K of a relation R has a mutable attribute, and let $C = (K \cap Fz)^+$. Since K is minimal, $C \neq U$, and therefore $PK \not\subseteq C$. Let $Z = (C \cap Fz) \cup N$. Then $Z \supseteq K$, so $Z^+ = U$, whereas $PK \not\subseteq Z$ because $PK \subseteq Fz$. Consequently, Z is not closed, and some elementary FD $Y \to b$ in Σ^+ has $Y \subseteq Z$ and $b \notin Z$. By condition (4), $Y \subseteq Fz$, so $Y \subseteq C$ and $b \in C$. Since $N \subseteq Z$, also $b \in Fz$, and thus $b \in C \cap Fz \subseteq Z$, a contradiction. □

Proof of Thm. 3.5. Since Σ contains no INDs, $M^* = Fz$.

(4)⇒(3). Let $X \to A$ be an elementary FD in Σ^+. If X is a superkey, it contains a key that determines A, and since no proper subset of X determines A, X is itself a key. By (4), X is a frozen key, so the FD belongs to $\mathcal{G}$. Otherwise, $X \cup \{A\} \subseteq Fz$ by (4), and the FD belongs to $\mathcal{G}$ again. Every nontrivial FD $Y \to A$ of Σ follows from an elementary FD $X \to A$ of Σ^+ with $X \subseteq Y$, so $\mathcal{G} \models \Sigma$.

(3)⇒(2) is Thm. 3.4, and (2)⇒(1) holds because every prefix state is an intermediate state.

(1)⇒(4). Let $X \to A$ be an elementary FD over R in Σ^+ that violates (4). We build a modification update (I, J) in which R holds two tuples, every other relation is empty, and either intermediate state with one changed entity violates Σ. In each case, I holds two tuples whose agree set C is closed and differs from U, so they differ on PK and are distinct entities. *Case 1: X is a superkey.* Then X is a minimal key with a mutable attribute, as shown above. Let $C = (X \cap Fz)^+$, and let J arise by swapping the mutable parts. Both entities change, since a mutable attribute of X lies outside C by the minimality of X. The intermediate state $\{t_1', t_2\}$ agrees on $(C \cap Fz) \cup N \supseteq X$ but not on PK, which violates the key X. *Case 2: X is not a superkey and A is mutable.* Let $C = X^+$, and let J give both tuples the same new value on A. The agree set of J is C again, and that of $\{t_1', t_2\}$ is $C \setminus \{A\} \supseteq X$, which violates $X \to A$. *Case 3: X is not a superkey, A is frozen, and some $x \in X$ is mutable.* Let $C = (X \setminus \{x\})^+$. Since the FD is elementary, $A \notin C$ and $x \notin C$. Let J arise by swapping the mutable parts, which changes both entities because $x \notin C$. The intermediate state $\{t_1', t_2\}$ agrees on $(C \cap Fz) \cup N \supseteq X$ but not on the frozen attribute A. The three cases cover every violation of (4), and the intermediate state $\{t_1, t_2'\}$ is symmetric in each. Hence the update is not executable. □

Each case of the last part changes exactly two entities, which must become visible together, so the proof yields the dichotomy stated after Thm. 3.5.

The proof also yields the polynomial test claimed in Thm. 3.5. For a set Z of attributes of R, let $Z' = Z \cup ((Z \cap Fz)^+ \cap Fz)$. The closure of Z under $\mathcal{G}$ is U if $(Z \cap Fz)^+ = U$, and Z' otherwise. Indeed, in the first case $Z \cap Fz$ contains a frozen key, and in the second case Z' is closed under both kinds of FDs in $\mathcal{G}$, since $(Z' \cap Fz)^+ = (Z \cap Fz)^+$ contains no key. Deciding $\mathcal{G} \models \Sigma$ takes one closure under Σ for every FD of Σ.

For fresh updates, we use that the left sides of elementary FDs consist of attributes that occur on left sides of Σ.

Proof of Prop. 3.7. Let $X \to A$ be an elementary FD in Σ^+, and let $x \in X$ occur on no left side of an FD in Σ. Then x fires no FD of Σ, so the closure of X is the closure of $X \setminus \{x\}$ together with x, and $A \neq x$ lies in the closure of $X \setminus \{x\}$, which contradicts elementarity. Hence every attribute of X receives only new values from a fresh update, unless the update leaves it unchanged.

Assume the condition, and let the tuples of two distinct entities violate Σ in an intermediate state $I \oplus T$. They violate an elementary FD $X \to A$ of Σ^+, and since I and J satisfy Σ, one of them is the new tuple $J(u)$ of some $u \in T$ and the other the old tuple $I(v)$ of some $v \in Ch \setminus T$. For every $x \in X$, the update leaves $J(u)[x] = I(u)[x]$, since a new value of x would differ from $I(v)[x]$. Hence $I(u)$ and $I(v)$ agree on X, so X is no superkey, and they agree on A, so $J(u)[A] \neq I(u)[A]$ and A is mutable, which contradicts the condition. Consequently, every fresh update is order-insensitive.

Conversely, let $X \to A$ be an elementary FD in Σ^+ whose left side is no superkey and whose right side is mutable. Case 2 of the proof of Thm. 3.5 takes two tuples whose agree set is X^+ and gives both a value of A that occurs nowhere in I. The update writes only this value, so it is fresh, and neither intermediate state with one changed entity satisfies $X \to A$. □

A.3 Proofs for Sec. 4

Order-sufficiency inherits the undecidability of implication.

Proposition A.6 (Undecidability). *For FDs and INDs, it is undecidable whether Σ is order-sufficient under Fz, and whether Σ is order-insensitive under Fz, even when the workload modifies a single attribute and the frozen attributes are exactly the prime attributes.*

The proof needs a source problem without FDs whose left side is empty.

Proof of Prop. A.6. Finite implication of an FD by FDs and INDs remains undecidable when every FD has a nonempty left side, as Abiteboul et al. note in their proof of this result [1, p. 200]. Let Σ be such a set over relations with *original* attributes, and let $X \to A$ be an FD over R with $A \notin X$. We may assume $X \neq \emptyset$. Indeed, the instance that holds one tuple of zeros in every relation satisfies Σ, and so does its disjoint union with a copy of ones, since FDs with nonempty left sides and INDs survive disjoint unions. Hence Σ implies no nontrivial FD with an empty left side, and the restriction to $X \neq \emptyset$ keeps the problem undecidable.

We add to every relation S a surrogate primary key ID_S, add to R a new attribute V with the FD $XV \to A$, and freeze every attribute except V. Call the result Σ'. Assume first that $\Sigma \models X \to A$. Every intermediate state L of a modification update (I, J) agrees with I outside V. It satisfies every constraint of Σ' that does not mention V, as I does, and its projection on the original attributes satisfies Σ and therefore $X \to A$. Consequently, L satisfies $XV \to A$, and Σ' is order-insensitive. Assume next that $\Sigma \not\models X \to A$, and let I_0

be a finite instance of Σ with tuples t_1 and t_2 that agree on X but not on A. We extend I_0 with distinct surrogate values, give t_1 and t_2 the V-values 1 and 2, and give every other tuple a new V-value. The result I satisfies Σ', and so does the instance J that swaps the V-values of t_1 and t_2. Either intermediate state with one changed entity makes t_1 and t_2 agree on XV but not on A. Hence Σ' is order-sufficient, and order-insensitive, exactly when $\Sigma \models X \to A$, which shows the first claim.

For the second claim, we add to every relation S and every original attribute B of S a frozen attribute N_B with the FD $BN_B \to ID_S$, and to R the FDs $XN_C \to ID_R$ for every original attribute C of R. The argument above carries over, since the new FDs do not mention V and pairwise distinct N-values satisfy them. It remains to show that the prime attributes are exactly the frozen ones. The surrogate ID_S is a key. Every original B and its N_B form a minimal key, for two reasons. Firstly, B alone is no superkey, as witnessed by two tuples that agree on all original attributes and on V and differ elsewhere. Secondly, N_B alone is no superkey, as witnessed by the disjoint union above, extended with equal N-values and distinct V-values: its two tuples per relation differ on every original attribute, which satisfies the new FDs because $X \neq \emptyset$. Finally, V is not prime. The first witness shows that every superkey contains a surrogate or an N-attribute. Let K be a minimal key of R with $V \in K$. Then K contains no surrogate, since the surrogate alone is a key, and hence some N_C. Since $K \setminus \{V\}$ is no superkey, some instance has tuples $p \neq q$ that agree on $K \setminus \{V\}$, and hence on N_C. By $XN_C \to ID_R$, the tuples p and q differ on X. Giving every tuple of R a distinct V-value, except that q takes the value of p, preserves $XV \to A$ and every other constraint. The resulting instance has two tuples that agree on K, a contradiction. □

Unary INDs. The first class of Sec. 4.1 allows only INDs $R[A] \subseteq S[B]$ with single attributes A and B, which Cosmadakis et al. [22] call *unary*. Unary INDs and FDs interact through cycles, which can make an attribute a key in every finite instance (Ex. A.9). Armstrong's axioms, transitivity of unary INDs and such cycle rules axiomatize finite implication, which is decidable in polynomial time [22], [1, Thm. 9.4.7]. The following theorem shows that finite implication decides order-sufficiency for unary INDs.

Theorem A.7 (Exactness for unary INDs). *Let Σ consist of FDs with nonempty left sides and unary INDs. Conditions (1) to (3) are equivalent to (4) every nontrivial unary IND in Σ^+ has a frozen right side, and every elementary FD $X \to A$ in Σ^+ satisfies $X \subseteq Fz$, and also $A \in Fz$ if X is not a superkey, and to (5) no modification update has two changed entities that must become visible together. Condition (4) is decidable in polynomial time.*

For unary INDs, the key step permutes values column by column.

Lemma A.8. *Let Σ consist of FDs and unary INDs, let I satisfy Σ, and let R be a relation. For every mutable attribute B of R, let π_B be a permutation of the values in column B of $I(R)$. Replacing every tuple t of R by the tuple that agrees with t on $Fz(R)$ and takes the value $\pi_B(t[B])$ on every mutable B yields an instance J, and (I, J) is a modification update.*

Proof. Entities and frozen values are unchanged. Every π_B is injective, so every two tuples of R keep their agree set, and J satisfies the FDs. Every column keeps its set of values, and J satisfies the unary INDs. □

Proof of Thm. A.7. Since $Fz \subseteq M^*$, a unary IND with a frozen right side belongs to $\mathcal{G}$.

(4)⇒(3). The first part of (4) places every nontrivial unary IND of Σ^+ in $\mathcal{G}$. The second part places every elementary FD of Σ^+ in $\mathcal{G}$, as in the proof of Thm. 3.5. Hence $\mathcal{G} \models \Sigma$. Thm. 3.4 gives (3)⇒(2), and (2)⇒(1) is immediate. (1)⇒(5) holds because every write order passes through a prefix state that contains the new tuple of exactly one of two such entities.

(5)⇒(4). Assume that (4) fails. In each of the following cases, we take a finite instance I of Σ, obtain J by Lem. A.8, and exhibit two changed entities that must become visible together. Every intermediate state that contains the new tuple of exactly one of them violates a constraint of Σ^+, and hence Σ. We describe the state in which the first entity is new, and the other state is symmetric. We use that the disjoint union of an instance of Σ with a copy in which all values are renamed is again an instance of Σ, since the FDs have nonempty left sides. *Case 1: a nontrivial unary IND $R[A] \subseteq S[B]$ in Σ^+ has a mutable right side.* Let I be the disjoint union of the instance with one tuple of zeros per relation and its copy with $0'$. Transpose 0 and $0'$ in column $S.B$. The changed entities are the two tuples of S. If only the first is new, column $S.B$ holds only $0'$, whereas column $R.A$ still holds 0. *Case 2: some elementary FD $X \to A$ in Σ^+ has a superkey X with a mutable attribute.* Then X is a minimal key, and $X \cap Fz$ is no superkey. Let I hold distinct tuples t_1 and t_2 of R that agree on $X \cap Fz$, and transpose $t_1[B]$ and $t_2[B]$ in every mutable column B. Both tuples change, since they differ on a mutable attribute of the key X. The new t_1 agrees with the old t_2 on X but not on PK, which violates the key X. *Case 3: some elementary FD $X \to A$ in Σ^+ has a non-superkey X and a mutable A.* Let I hold distinct tuples t_1 and t_2 that agree on X, and hence on A with a value a, together with a renamed copy that carries a value b in column A. Transpose a and b in column A. The new t_1 agrees with the old t_2 on X but not on A. *Case 4: some elementary FD $X \to A$ in Σ^+ has a non-superkey X with a mutable x, and A is frozen.* Since $X \setminus \{x\}$ does not determine A, let I hold tuples t_1 and t_2 that agree on $X \setminus \{x\}$ but not on A, and transpose $t_1[B]$ and $t_2[B]$ in every mutable column B. Both tuples change, since they differ on x, as X determines A. The new t_1 agrees with the old t_2 on X but not on A. The four cases cover every violation of (4).

Polynomial time. By the axiomatization of finite implication for FDs and unary INDs, its cycle rules derive only unary FDs and unary INDs [1, Thm. 9.4.7]. Consequently, the FDs in Σ^+ are generated by the FDs of Σ and polynomially many unary FDs, all computable in polynomial time [22]. The first part of (4) inspects polynomially many unary INDs, and the second part is the test after the proof of Thm. 3.5, applied to these generators. □

The following example shows why condition (4) must be computed under finite implication.

Example A.9 (Finite implication matters). Let $R(K, X, A)$ have the primary key K, the unary IND $R[K] \subseteq R[X]$ and the declared FD $X \to A$, and let $Fz = \{K, X\}$. In every finite instance, X takes at least as many values as K, so X is a key, condition (4) holds, and every modification of A is order-insensitive. Under unrestricted

implication, X is not a key, and the test would wrongly reject the schema.

For the class $\mathcal{A}$, the proof of Thm. 4.1 completes two tuples to a legal instance by the following chase.

Lemma A.10. *Let F be a set of FDs over a relation S, let K be a minimal key of S under F, and let V be a finite set of values for K. Some finite relation over S satisfies F and has exactly V as its projection on K.*

Proof. For every value in V, create a tuple that takes this value on K and distinct nulls elsewhere. Chase with F: whenever two tuples agree on Z and $Z \to E \in F$, merge their values on E. For tuples u and v, let $M(u,v)$ be the attributes outside K on which they carry merged nulls, and let $A(u,v)$ be the attributes of K on which they agree. We show the invariant that $K \setminus \{B\} \to M(u,v)$ follows from F for all u, v and $B \in K$ with $u[B] \neq v[B]$. It holds initially, since M is empty. Let a step merge the E-values of p and q, which agree on Z with $Z \to E \in F$, and let it newly merge x and y, where x was merged with p and y with q on E before. If $x[B] \neq y[B]$, two neighbors in the sequence x, p, q, y differ on B. For the pairs x, p and q, y, the invariant already gives $K \setminus \{B\} \to E$. For the pair p, q, we have $Z \subseteq A(p,q) \cup M(p,q)$ and $A(p,q) \subseteq K \setminus \{B\}$, so the invariant gives $K \setminus \{B\} \to Z$ and therefore $K \setminus \{B\} \to E$. Now assume that the chase must merge two distinct values on some $B \in K$, because p and q agree on Z with $Z \to B \in F$ and $p[B] \neq q[B]$. The invariant gives $K \setminus \{B\} \to Z$, hence $K \setminus \{B\} \to B$, which contradicts the minimality of K. Hence the chase never merges values of K, and replacing every class of nulls by a new value yields the relation. □

Proof of Thm. 4.1. For a relation R, let F_R be its declared FDs.

(4)⇒(3). By Lem. A.5, applied to every F_R, every minimal key of every relation is frozen. The right side of every IND is such a key, so every IND belongs to $\mathcal{G}$. As in the proof of Thm. 3.5, every elementary FD of F_R^+ belongs to $\mathcal{G}$, since a key under F_R is a key under Σ. Consequently, $\mathcal{G} \models \Sigma$. Thm. 3.4 gives (3)⇒(2), and (2)⇒(1)⇒(5) is immediate.

(5)⇒(4). Assume that F_R violates condition (4) of Thm. 3.5. The proof of that theorem yields tuples t_1, t_2 and t_1', t_2' over R such that both pairs satisfy F_R and either intermediate state with one changed entity violates F_R. We complete both pairs with the same tuples W. Process the relations that the IND graph reaches from R in topological order. For such a relation S, collect the values that the tuples built so far, including t_1, t_2, t_1', t_2', carry on the left sides of the INDs into S. All these INDs reference the same minimal key of S, so Lem. A.10 builds the tuples of S. The process terminates because the IND graph is acyclic, and it adds no tuple to R. All other relations stay empty. Then $\{t_1, t_2\} \cup W$ and $\{t_1', t_2'\} \cup W$ satisfy Σ, contain the same entities, and agree on frozen values. This update has exactly two changed entities and is not executable. □

The running example illustrates Thm. 4.1.

Example A.11 (Without the cached path). Without the cached column *gpath*, Ex. 1.1 keeps a foreign key to a primary key, so the schema belongs to $\mathcal{A}$. Thm. 4.1 then makes every modification order-insensitive, renames and moves included. It is the cached copy, not the foreign key, that defeats write order in Ex. A.2.

For the class $\mathcal{K}$, both directions work IND by IND.

Proof of Thm. 4.3. (4)⇒(3). The primary keys are frozen keys and belong to $\mathcal{G}$. Let $R[PD] \subseteq S[LC]$ satisfy (4). If $C \subseteq Fz$, its right side lies in $Fz \subseteq M^*$. Otherwise, $P \subseteq Fz$ and every D_i is frozen or copies a frozen C_i. In the latter case, the rule of Def. 3.1 puts D_i into $M^*(R)$, because L is frozen and P sits at its positions. The left side then lies in $M^*(R)$, the right side contains the frozen key L, and the IND belongs to $\mathcal{G}$. Hence $\mathcal{G} \models \Sigma$. Thm. 3.4 gives (3)⇒(2), and (2)⇒(1) is immediate.

(1)⇒(4). Let $\rho = R[PD] \subseteq S[LC]$ violate (4). Then $C \not\subseteq Fz$, and either some position i has a mutable C_i and a mutable D_i, or $P \not\subseteq Fz$ and every mutable C_i has a frozen D_i. Note that $R \neq S$, since the IND graph is acyclic. Let π, π_1, π_2, a and b be new constants. *Core of the first case.* An entity r of R points to π, and the entity s of S with key π has $s[C_i] = r[D_i] = a$ in I and b in J. All other copies and sources of ρ are 0. Either intermediate state with one changed entity leaves r without a witness. *Core of the second case.* Choose a mutable C_j, whose copy D_j is frozen. Two entities r_1 and r_2 of R differ on an attribute of $PK(R)$ outside P, which exists because P does not contain $PK(R)$. Their pointers π_1 and π_2 differ exactly on the mutable attributes of P, and $r_1[D_j] = a$ and $r_2[D_j] = b$. The entities s_1 and s_2 of S with keys π_1 and π_2 have $s_1[C_j] = a$ and $s_2[C_j] = b$. In J, r_1 points to π_2, r_2 points to π_1, $s_1[C_j] = b$ and $s_2[C_j] = a$. All other copies and sources of ρ are 0. Whichever of the four entities is written first, some entity of R loses its witness: a new r_1 or r_2 finds no matching tuple yet, and a new s_1 or s_2 strips the old r_1 or r_2. *Completion.* The constants π, π_1 and π_2 have pairwise distinct new components, and the primary keys of core tuples consist of new constants. Every other attribute of a core tuple that occurs in a pointer gets a new constant, and every remaining attribute gets 0. Every relation gets a base tuple of zeros. For every core or support tuple and every IND $Q[P'D'] \subseteq T[L'C']$ of its relation other than ρ, the relation T needs a witness whose key is the value of the tuple on P' in I or J. Unless a core tuple has this key, T gets a support tuple with this key and 0 elsewhere, and the process terminates because the IND graph is acyclic. Such a support tuple is a witness, since the copies of the referencing tuple are 0 and the sources of T lie outside its key. Core tuples lie in R and S, and support tuples never reach R. A tuple can need the key π, π_1 or π_2 of S only through an IND without copies. Indeed, the other pointers of R into S differ from P in some position, and a support tuple carries 0 outside its key, so the pointer of an IND with copies, which leaves the primary key, yields a key with a component 0. The core tuple with this key is then a witness, since there are no copies to match. Hence every relation keeps distinct keys, every core violation persists, and the update is not executable. □

Both halves of restriction (iv) on the pointers of class $\mathcal{K}$ are needed. Firstly, let the relation $R(A, B, D)$ have the primary key A, let $S(K_1, K_2, C)$ have the primary key K_1K_2, and let only B and C be mutable under $R[ABD] \subseteq S[K_1K_2C]$, whose pointer contains $PK(R)$. Condition (4) fails, but an entity of S can only be referenced by the entity of R whose key it carries. Hence writing first the entities of S that gain their referencing entity, then the entities of R, and last the other entities of S executes every modification update. Yet an update that moves an entity of R away from an entity of

S and changes that entity is not order-insensitive. Secondly, let $T(K_1, K_2, V)$ be frozen with the primary key K_1K_2, let $R(K, A, B, D)$ have the primary key K and the frozen attributes B and D, and let $S(L, E)$ have only its primary key L frozen. The INDs are $R[AD] \subseteq S[LE]$, $R[AB] \subseteq T[K_1K_2]$ and $T[K_1V] \subseteq S[LE]$, and the pointer of the third lies inside $PK(T)$. Condition (4) fails for the first IND, since A and E are mutable. Yet the key L makes the three INDs imply $R[ABD] \subseteq T[K_1K_2V]$, whose right side is frozen, and this IND and the third one imply the first. Hence $\mathcal{G} \models \Sigma$, and the schema is order-insensitive by Thm. 3.4. As in Ex. A.14 below, the frozen relation T fixes the values that referenced entities of S carry.

For fresh updates, executability needs only the second half of condition (4), and the proof orders the relations along the IND graph.

Proof of Prop. 4.5. Let (I, J) be a fresh modification update, let $\rho = R[PD] \subseteq S[LC]$ be an IND of Σ, and let r be an entity of R. Let s_I and s_J be the entities of S whose keys are $I(r)[P]$ and $J(r)[P]$. They exist and are unique, since L is a frozen primary key, and every intermediate state needs one of them as the witness of r under ρ.

Assume that every position i has $C_i \in Fz$ or $D_i \in Fz$. Then s_J keeps its sources. Indeed, for a mutable C_i the copy D_i is frozen, so $J(s_J)[C_i] = J(r)[D_i] = I(r)[D_i]$ is a value of C_i in I, which a fresh update writes into no source. Hence the new tuple of r has its witness in every intermediate state, and the old tuple of r lacks it exactly when s_I is new and has changed a source of ρ. Since the primary keys are frozen and are the only FDs, an intermediate state $I \oplus T$ is legal exactly when T contains every such r whenever it contains s_I. Each of these requirements leads from an entity of R to an entity of S along an edge of the IND graph, so writing the relations in a topological order of the IND graph, referencing relations first, keeps every prefix state legal. If condition (4) of Thm. 4.3 holds as well, every modification update is order-insensitive by that theorem.

Conversely, let some position i have a mutable C_i and a mutable D_i. The core of the first case in the proof of Thm. 4.3, completed as there, changes only r and s and writes the new constant b, so it is a fresh update with two changed entities that is not executable. Finally, let every position satisfy the condition, and let condition (4) of Thm. 4.3 fail for ρ. Then $P \not\subseteq Fz$ and some C_j is mutable. An entity r of R points to π_1 in I and to π_2 in J, where π_1 and π_2 differ exactly on the mutable attributes of P, and the entities s_1 and s_2 of S with these keys have $s_1[C_j] = s_2[C_j] = r[D_j] = a$ in I. Besides r, only s_1 changes, to $s_1[C_j] = b$, and all other copies and sources of ρ are 0. After the same completion, the update is fresh, since b and π_2 are new, and writing s_1 before r strips the old r of its witness. □

A bounded SAT search confirms Prop. 4.5, including the order of the relations, on 17,436 pairs of 403 random schemas of class $\mathcal{K}$ and freezes, with up to five entities and five values, and finds a witness with two changed entities wherever a condition fails.

Thm. 4.6 combines Props. 3.7 and 4.5 under the freeze that a template induces.

Proof of Thm. 4.6. Let M be the attributes that the template modifies. Its updates are the fresh modification updates under the freeze Fz_M of all attributes outside M, and it needs atomic visibility exactly when one of them is not executable. If Σ contains only FDs, consider the elementary FDs $X \to A$ in Σ^+ whose left side X is not a superkey. By Prop. 3.7 under Fz_M, all updates of the template are order-insensitive, and hence executable, if M contains no right side of these FDs, and otherwise one of them is not executable. If Σ belongs to $\mathcal{K}$, Prop. 4.5 under Fz_M shows that all of them are executable if and only if no IND $R[PD] \subseteq S[LC]$ in Σ has both $C_i \in M$ and $D_i \in M$ at some position i. □

Unique keys. Unique keys besides the primary key can pin pointers. Indeed, if a referencing entity holds frozen copies of attributes that form a unique key of S, then no fresh update can move it to another entity of S, since that entity would have to take over values that are not new in their columns. Let Σ belong to $\mathcal{K}$, except that its relations may also declare unique keys, which no IND references. For a set M of attributes and an IND $\rho = R[PD] \subseteq S[LC]$, let $X_\rho(M)$ be the least set of attributes of S that contains every attribute of LC whose counterpart in PD lies in M, and the copies of every IND $S[P'D'] \subseteq T[L'C']$ with $P' \subseteq X_\rho(M)$. Let Fz^* be the least set $M \supseteq Fz$ that contains the pointer P of every IND ρ for which $X_\rho(M)$ contains a unique key of S. It exists, since $X_\rho(M)$ grows with M, and it results from adding such pointers to Fz until none is missing. The iteration matters when pointers overlap. Once Fz^* contains the pointer of ρ, the attributes that it shares with the pointer of another IND ρ' out of R enter M, and their counterparts may complete a unique key in $X_{\rho'}(M)$.

Proposition A.12 (Unique keys). *For such schemas, Prop. 4.5 holds with Fz^* in place of Fz.*

Hence the test of Thm. 4.6 (b) stays valid under unique keys once a template counts the pointers that Fz^* adds as unmodified, as Sec. 4.2 states.

Proof. Let (I, J) be a fresh modification update under Fz. Firstly, every intermediate state satisfies the unique keys. Indeed, let u and v be distinct entities of a relation with a unique key U, and consider the old tuple of u next to the new tuple of v. If v keeps its values on U, the two tuples differ on U since I is legal, and otherwise v writes into U a value that is new in its column, which u does not carry in I. Secondly, (I, J) changes no attribute of Fz^*. We show by induction along the iteration that if (I, J) changes no attribute of a set $M \supseteq Fz$, then it changes no pointer that the next step adds. Indeed, let $X_\rho(M)$ contain a unique key U of S, let r be an entity of R, and let s_I and s_J be the entities of S whose keys are $I(r)[P]$ and $J(r)[P]$. Let A be an attribute of LC whose counterpart B in PD lies in M. Then $J(s_J)[A] = J(r)[B] = I(r)[B] = I(s_I)[A]$. If A lies in the key L, which is frozen, then $I(s_J)[A] = J(s_J)[A]$. If A is a source, then $J(s_J)[A]$ is a value of A in I, so the fresh update has not written it, and again $I(s_J)[A] = J(s_J)[A]$. The copies of an IND $S[P'D'] \subseteq T[L'C']$ are determined by P', since L' is the primary key of T. Hence s_I and s_J agree on $X_\rho(M)$ in I, so on U, and $s_I = s_J$, so r keeps its pointer P. The fresh modification updates under Fz and under Fz^* thus coincide, and the unique keys hold in every intermediate state. Then the first part of the proof of Prop. 4.5 applies with Fz^* in place of Fz. If condition (4) holds for Fz^* as well, then either s_I and s_J keep their sources, or $s_I = s_J$ and every position i has C_i or D_i unchanged, so every combination of old and

new tuples of r, s_I and s_J satisfies ρ, and every intermediate state is legal.

Conversely, the counterexamples of the proof of Prop. 4.5, taken under Fz^*, are fresh modification updates under Fz, and they carry over once the completion gives every support tuple its own new constants outside its key and every copy the value of its source. Two tuples of one relation then agree on an attribute only where the construction forces them to. This happens only for the old and the new witness of an entity r that moves under some IND ρ whose pointer P is not contained in Fz^*. The two witnesses agree on the attributes of L whose counterparts in P lie in Fz^*, on the sources whose copies lie in Fz^*, and on the copies that these attributes determine, that is, on $X_\rho(Fz^*)$, and on nothing else. Since Fz^* is a fixpoint, $X_\rho(Fz^*)$ contains no unique key of S. Hence the two witnesses differ on an attribute of every unique key, and every counterexample respects the unique keys. □

The bounded SAT search confirms Prop. A.12 on 4,172 pairs of random schemas and freezes with one or two unique keys of one or two non-key attributes, where six pairs need seven entities for their witness. Under Fz in place of Fz^*, 157 of these pairs would be predicted wrongly. Pointers that overlap need the iteration. The search also covers 8,498 pairs of 144 schemas in which two INDs out of one relation share pointer attributes and the unique keys lie on sources, on sources and key attributes, or on copies. It finds a counterexample with up to eight entities for every pair that Prop. A.12 rejects, and none with up to five for the others. Computing every set X_ρ under Fz alone, without the iteration, rejects 28 pairs wrongly.

FDs together with class $\mathcal{K}$. Schemas that combine FDs with class $\mathcal{K}$ lie outside both cases of Thm. 4.6. When no template modifies the right side of an FD of Σ whose left side is not a superkey, the test of case (b) decides the schema. Indeed, a fresh update writes only new values into the left side of such an FD and leaves its right side unchanged, so the FD holds in every intermediate state, as in the proof of Prop. 3.7. GitLab is such a case, since these right sides are its frozen sharding keys (Sec. 6.4). The general combination is open.

Conditional INDs. The proof of Thm. 4.7 reads the precedence graph of Prop. A.1 through enabled and disabled entities.

Proof of Thm. 4.7. The argument of Prop. A.1 applies to CINDs, since a condition only selects the tuples that need a witness and the tuples that can serve as one, and $L = PK(S)$ identifies one entity. Hence the update is binary. An edge from v to u either leaves a referenced entity v whose new tuple alone serves the new tuple of u, or enters a referenced entity u whose old tuple alone serves the old tuple of v. The update then enables v in the first case and disables u in the second. No update both enables and disables an entity e of S. Indeed, both tuples of e would be present, with the old value of g in $B_2 \setminus B_1$ and the new one in $B_1 \setminus B_2$ for the sets B_1 and B_2 of two CINDs into S, but one of these differences is empty, since the sets form a chain. Every edge thus leaves an enabled entity or enters a disabled one, and it joins two relations that the acyclic IND graph orders strictly. Consider the order of the theorem and an edge from v to u. If the edge leaves an enabled v, then v lies in the first group, and u lies in a later group or is an enabled entity of a relation that references the relation of v, which the first group writes later. Symmetrically, if the edge enters a disabled u, then u lies in the last group, after every entity of the earlier groups and after the disabled entities of the relations that reference the relation of u. No edge joins two entities of the middle group. Hence the order respects every edge, every prefix is closed under predecessors, and every prefix state is legal. □

Nullable foreign keys are CINDs as well: a nullable foreign key is the CIND whose condition on R is that the pointer is not null. Under modifications it enables and disables nothing, and setting the pointer to null or to an existing parent changes one entity, so such a change needs no order.

Order-insensitivity on C has a characterization as well, under two restrictions on pointers: (a) every CIND with a condition on S has a pointer $P \not\subseteq PK(R)$, and (b) if such a CIND has no condition on R, it shares its pointer and S with no other CIND.

Proposition A.13 (Order-insensitivity under conditional INDs). *Let Σ belong to C and satisfy restrictions (a) and (b). Then every modification update is order-insensitive under Fz if and only if every CIND that tests a mutable attribute g has a frozen pointer P and tests no mutable attribute f.*

In the example of the flag *arch* in Sec. 4.3, the schema is order-insensitive only if the workload never archives groups, or neither moves nor archives projects. Without the restrictions, a CIND can imply another, and the condition remains sufficient but is no longer necessary. For (a), let $R[P] \subseteq T[K]$ and $T[K] \subseteq S[L; g \in B]$ with $K = PK(T)$ imply $\rho = R[P] \subseteq S[L; g \in B]$, and let P and g be mutable. The condition fails for ρ, yet every entity of S that an entity of R points to in I or J shares its key with an entity of T in both instances. The second CIND then makes it meet $g \in B$ in both, so every modification update is order-insensitive. For (b), let $Project[gid] \subseteq Group[gid; arch \in \{0\}]$ join the CIND of Sec. 4.3, which it implies, and let $Project.gid$ be frozen while both flags are mutable. The condition fails for the CIND of Sec. 4.3, which tests the mutable flag of projects, yet every group with a project keeps $arch = 0$ in I and in J, so every modification update is order-insensitive. The proof also covers fresh updates, where on CINDs a modification update is *fresh* if, beyond Sec. 3.3, every value that it writes into an attribute that a CIND tests on its right side is new in its column.

Proof of Prop. A.13. Let every CIND that tests a mutable g have a frozen pointer and test no mutable f. A modification keeps every entity present, so a CIND that tests a frozen g or no attribute of S yields no edge. For a CIND that tests a mutable g, a tuple of R that needs a witness in I needs it in J at the same entity, and conversely, so this entity meets the condition on S in both versions, and the CIND yields no edge either. Hence the precedence graph has no edge, and every intermediate state is legal. This direction does not use the restrictions.

Conversely, let $\rho = R[P; f \in A] \subseteq S[L; g \in B]$ test a mutable g, and let $P \not\subseteq Fz$ or f be mutable. For every relation whose CINDs test an attribute on the right side, fix a value b_1 in the least set of its chain, and for S also a value b_0 outside every set of its chain. The update changes two entities, r of R and s of S, where s carries $g = b_0$ in I and $g = b_1$ in J, and both keep all other values. If $P \not\subseteq Fz$, then r meets $f \in A$, if ρ tests f, in I and J, and it points to s in J and to an unchanged entity s' with $g = b_1$ in I, whose key differs

from that of s exactly at the mutable attributes of P. If $P \subseteq Fz$, then f is mutable, r points to s in I and J, and f changes from a constant outside every set that a CIND tests f against to a value in A. The keys of s and s' and every other attribute of r, s and s' that no CIND tests carry new constants, and every other tested attribute on the left carries a constant outside all sets of its CINDs. Top-down along the acyclic IND graph, the completion then gives every tuple that needs a witness in I or J an unchanged entity at the key that it points to, with b_1 in the tested attribute, if any, and new constants in its other attributes. Hence every pointer of a completion entity into S under a CIND with a condition on S contains an attribute outside the key by restriction (a), whose value differs from the new constants in the key of s, so it never points to s.

I and J are legal. Every completion entity and s' meet every condition on their right side, since b_1 lies in every set of their chains, and they exist in both instances. The new tuple of s meets every condition on S as well. In J, the new tuple of r needs at s at most these conditions, and in I it points to s' if $P \not\subseteq Fz$. If $P \subseteq Fz$, then the old tuple of r meets no condition on R, so it needs at s only the CINDs without a condition on R, which by restriction (b) test no attribute of S, since they share the pointer and S with ρ. Hence no tuple of I needs the old tuple of s to meet a condition on S, and every other demand of I and J is met by construction. The intermediate state that shows only r new is illegal, since the new tuple of r meets $f \in A$ and points to the old tuple of s, which fails $g \in B$. The update is thus not order-insensitive.

The reverse update (J, I), with b_0 new in the column of g and, if $P \subseteq Fz$, the constant of f in I new in its column, is fresh, since it writes no key and no other tested value. It disables s, and its intermediate state that shows only s new is illegal, so the proposition also holds for fresh updates. □

For modification updates, both proofs use only that L identifies one entity in every state. Both results therefore hold even when L is a unique key of S that no update modifies, as for the flags that Sentry sets for accounts provisioned by an identity provider, one of the one-way conditions of Sec. 6.2.

Beyond the exact classes. Outside the three classes, write order can suffice for INDs although the condition of Thm. 3.4 fails, as the following example shows for frozen reference data.

Example A.14 (Frozen reference data). Let $Ref(L, Q)$ be a reference table with primary key L that the workload never modifies. Let $R(N, P, D)$ have the primary key N, the mutable pointer P and the frozen copy D, and let $S(L, C)$ have the primary key L and the mutable source C. The INDs $R[P, D] \subseteq Ref[L, Q]$ and $R[P, D] \subseteq S[L, C]$ require every P-value to lead to a tuple of Ref and a tuple of S that both carry the D-value. Since L is a key of Ref and Ref never changes, the C-value that a referenced entity of S must carry is fixed by its key. Hence every modification update is executable: write first the entities of S that gain referencing entities, then the entities of R, and last the entities of S that lose them. Writing first an entity of S that loses its referencing entities strips them of their witness, so such updates are not order-insensitive. The schema is order-sufficient but not order-insensitive, and $\mathcal{G} \not\models \Sigma$.

Two INDs into one relation can force a cycle of four writes although each is harmless alone. Let $T(P, V)$ be a frozen reference table with key P, let $R(K, A, D)$ and $Q(L, B, G)$ have frozen keys and the frozen attributes D and G, and let $S(C, E, F)$ have the frozen key C. The INDs are $R[AD] \subseteq T[PV]$, $Q[BG] \subseteq T[PV]$, $R[AD] \subseteq S[CE]$ and $Q[BG] \subseteq S[CF]$. Each IND into S alone behaves as in Ex. A.14. Together, let T hold $(1, 0)$ and $(2, 0)$, let r move from 1 to 2 and q from 2 to 1, and let s_1 and s_2 with keys 1 and 2 change (E, F) from $(0, 1)$ to $(1, 0)$ and from $(1, 0)$ to $(0, 1)$. Then r waits for s_2, s_2 for q, q for s_1, and s_1 for r, so no order exists. We conjecture that the condition $\mathcal{G} \models \Sigma$ remains exact for acyclic INDs if no pointer contains the primary key of its own relation, no IND derives a key, and no IND transfers an FD from frozen reference data. The evidence for this conjecture compares the condition with a semantic check that enumerates small modification updates, where a SAT search over larger updates covers the cases in which the condition fails but enumeration finds no cycle. A random generator combines composite pointers, pointers that meet the primary key, attributes shared by two INDs and chains of copies. On its schemas, the semantic check confirms the decider of class $\mathcal{K}$ on all 33,445 pairs inside the class and agrees with the condition on all 5,400 pairs in the class of the conjecture. Outside this class, the condition fails while write order suffices on 1,039 of 46,556 pairs. In 1,029 of them some update is not order-insensitive, and each has a pointer that contains the primary key of its relation or ranges over frozen reference data. In the other 10, a frozen copy fixes the attribute it copies in every referenced entity, so every update is order-insensitive although $\mathcal{G} \not\models \Sigma$.

A.4 Proofs for Sec. 5

The conflict graph rewrites condition (4) of Thm. 4.3 as a set of binary clauses.

Proof of Thm. 5.1. Freezing $Z \subseteq W$ leaves the attributes in $W \setminus Z$ mutable. For an IND $R[PD] \subseteq S[LC]$, condition (4) of Thm. 4.3 is equivalent to the conjunction of the clauses "p or c is frozen" for all $p \in P$ and $c \in C$, and "C_i or D_i is frozen" for all positions i. Indeed, if $C \subseteq Fz$, every clause holds. Otherwise, some $c \in C$ is mutable, the first clauses freeze P, and the second clauses are the rest of condition (4). A clause that mentions an attribute outside W holds, and the other clauses are the edges of H. Hence Thm. 4.3 makes Σ order-sufficient exactly when Z covers every edge of H.

For the hardness, let G be a graph whose edges are oriented along a numbering of its nodes, so that the orientation is acyclic. For every node u, the relation R_u has the primary key K_u, the attribute A_u, and an attribute D_{uv} for every edge from u to v. Every such edge contributes the IND $R_u[A_u D_{uv}] \subseteq R_v[K_v A_v]$. The schema belongs to $\mathcal{K}$: the pointer A_u is shared by the outgoing edges of u and is disjoint from K_u, every copy D_{uv} lies outside K_u and occurs in one IND only, and every source A_v lies outside K_v and is no copy. With $W = \{A_u\}$ and unit weights, H has the edge $\{A_u, A_v\}$ for every edge of G and no other edge, since the copies are frozen. MIN-FREEZE with bound k then asks whether G has a vertex cover of size at most k, which is NP-hard [37]. □

The tractable case of Sec. 5.1 follows from the same graph. Every edge of H joins attributes of two relations that an IND connects, so a 2-coloring of the undirected IND graph colors H. A minimum-weight vertex cover of a bipartite graph is a minimum cut of the

Algorithm A.1 Freeze

Require: schema Σ in $\mathcal{K}$, modified attributes W with weights w
Ensure: a freeze $Z \subseteq W$ that makes Σ order-sufficient, or every fresh update executable, of minimum weight if H is bipartite and at most twice the minimum otherwise

1: $H \leftarrow$ the conflict graph of Σ on W, without the edges at pointers if only fresh updates must be executable
2: **if** H is bipartite with sides U and V **then**
3: link s to every $A \in U$ with capacity $w(A)$, every $B \in V$ to t with capacity $w(B)$, and A to B with capacity ∞ for every edge $\{A, B\}$ of H
4: $(S, T) \leftarrow$ a minimum s-t cut
5: **return** $(U \cap T) \cup (V \cap S)$
6: $r \leftarrow w$
7: **for all** edges $\{A, B\}$ of H **do**
8: $\varepsilon \leftarrow \min(r(A), r(B)), r(A) \leftarrow r(A) - \varepsilon, r(B) \leftarrow r(B) - \varepsilon$
9: **return** $\{A \in W : r(A) = 0\}$

standard flow network. The following example weighs the minimal freezes of the running example.

Example A.15 (Minimal freezes). In Ex. 4.4, H has an edge between *Project*.*gid* and *path* and one between *path* and *gpath*. Its minimal vertex covers are $\{path\}$ and $\{Project.gid, gpath\}$. Freezing *path* requires rewriting every rename, whereas freezing *gid* and *gpath* requires rewriting every move and every rename of a group with projects. Hence freezing *path* rewrites fewer updates unless renames of empty groups outnumber moves.

Alg. A.1 computes the minimum cut on a bipartite conflict graph (lines 2–5) and the 2-approximation of Bar-Yehuda and Even otherwise (lines 6–9).

For shard keys, sufficiency reuses Lem. A.4, and necessity reuses the two-tuple constructions of Thm. 3.5.

Proof of Thm. 5.2. *Sufficiency.* Write the shards one at a time, each in one atomic step and in any order. Assume that two tuples t and u violate an elementary FD $X \to A$ of Σ^+ in some intermediate state. By Lem. A.4, the FD is unstable. The tuples of one shard are all old or all new, and I and J satisfy Σ, so t and u lie on different shards. They agree on $X \cap Fz$, whose values never change, so their tuples in I agree on $X \cap Fz$ and hence on $(X \cap Fz)^+ \supseteq SK$. Since $SK \subseteq Fz$, the tuples t and u agree on SK and lie on the same shard, a contradiction.

Necessity. We first reduce the condition to two weaker ones. For every unstable elementary FD $X \to A$ in Σ^+, let (i) require $X^+ \supseteq SK$ if A is mutable, and let (ii) require $(X \setminus \{x\})^+ \supseteq SK$ for every mutable $x \in X$ if A is frozen. Assume (i) and (ii). We show $(X \cap Fz)^+ \supseteq SK$ by induction on $|X \setminus Fz|$, where an FD with a frozen right side precedes one with a mutable right side of the same size. Let $Y = X$ if A is mutable. Otherwise, A is frozen, X contains a mutable x because the FD is unstable, and we let $Y = X \setminus \{x\}$. Then $Y^+ \supseteq SK$ by (i) or (ii). Let $s \in SK$. If $s \in Y$, then $s \in X \cap Fz$ because $SK \subseteq Fz$. Otherwise, some $Y' \subseteq Y$ gives an elementary FD $Y' \to s$. If $Y' \subseteq Fz$, then $s \in (X \cap Fz)^+$. Otherwise, $Y' \to s$ is unstable, since a frozen key inside Y' would make $X \to A$ stable. It has a frozen right side and $|Y' \setminus Fz| \leq |Y \setminus Fz|$, so it precedes $X \to A$. The induction hypothesis gives $(Y' \cap Fz)^+ \supseteq SK$, and $s \in (X \cap Fz)^+$ because $Y' \cap Fz \subseteq X \cap Fz$.

It remains to refute (i) and (ii) by updates. If (i) fails, take two tuples whose agree set is $C = X^+$ and give both the same new value on A, as in Case 2 of the proof of Thm. 3.5. If (ii) fails, take two tuples whose agree set is $C = (X \setminus \{x\})^+$ and swap their mutable parts, as in Case 3. In both cases $SK \not\subseteq C$, so the two tuples differ on SK, and some placement puts them on different shards. Every execution by single-shard steps then passes through a state in which exactly one of them is new, and this state violates $X \to A$. □

For fresh updates, the condition of Thm. 5.2 weakens to $X^+ \supseteq SK$ and applies only to the FDs that Prop. 3.7 retains. Indeed, the proof of Prop. 3.7 shows that the two tuples of a violation agree on X already in I, and hence on X^+, so sufficiency needs $X^+ \supseteq SK$ only for these FDs, and necessity uses the construction for (i) alone.

Only unstable FDs constrain the shard key, and the condition uses closure and not containment, as the following example shows.

Example A.16 (Closure, not containment). Let a relation (*Id*, *Zip*, *City*, *Tax*) have the key *Id* and the FDs $Zip \to City$ and $City \to Tax$, and let only *Tax* be mutable. The shard key *City* meets the condition for the unstable FD $Zip \to Tax$, since tuples that agree on *Zip* lie in the same city.

On the running example, the condition selects the shard key as follows.

Example A.17 (Sharding projects by group). Consider again Ex. 3.6, where *Project* stores the path under $gid \to gpath$, and let the workload rename groups but never move projects. The FD $gid \to gpath$ is unstable, and the condition requires $\{gid\}^+ \supseteq SK(Project)$, so $SK(Project) = \{gid\}$, the only frozen attribute in $\{gid\}^+$. Sharding projects by their group turns every rename into one single-shard step, whereas sharding them by *pid* requires atomic visibility.

Class $\mathcal{K}$ yields binary updates, and the proof of the next theorem reads the edges of $\mathcal{P}$ off the INDs.

Proof of Thm. 5.3. Let (I, J) be a modification update, let $\rho = R[PD] \subseteq S[LC]$ be an IND of Σ, and let r be a changed entity of R. As in the proof of Prop. 4.5, let s_I and s_J be the entities of S whose keys are $I(r)[P]$ and $J(r)[P]$. The old tuple of r has a witness under ρ exactly when s_I is old or $J(s_I)[C] = I(r)[D]$, and the new tuple has one exactly when s_J is new or $I(s_J)[C] = J(r)[D]$. An unchanged entity of R keeps its witness, since its referenced entity has the same key in I and J, and I and J are legal. The primary keys are frozen and are the only FDs, so the update is binary, and the graph whose edges lead from some r to s_I with $J(s_I)[C] \neq I(r)[D]$ and from some s_J to r with $I(s_J)[C] \neq J(r)[D]$ is a precedence graph.

Sufficiency. Let ρ satisfy condition (4). If $C \subseteq Fz$, every entity of S keeps its sources, and the two inequalities fail because I and J are legal. Otherwise $P \subseteq Fz$, so $s_I = s_J$, and at every position i the source C_i or the copy D_i is frozen. In both cases $I(s_I)[C_i] = I(r)[D_i] = J(r)[D_i] = J(s_I)[C_i]$, and again neither inequality holds. Hence every edge of $\mathcal{P}$ stems from an IND that violates condition (4) and therefore carries the shard key. Its relations share a co-location group, and since I and J satisfy Σ, $I(r)[SK(R)] = I(s_I)[SK(S)]$ and $J(r)[SK(R)] = J(s_J)[SK(S)]$. Shard keys are frozen, so r, s_I and s_J agree on their shard-key values and lie on one shard under every placement. Every edge of $\mathcal{P}$ thus stays within one shard, so does every strongly connected component, and Lem. 5.5 (1) executes the update.

Necessity. Let ρ violate condition (4) and not carry the shard key. The update (I, J) of the proof of Thm. 4.3 for ρ has a cycle of $\mathcal{P}$ through an entity r of R and its witness s under ρ, namely $r \to s \to r$ in the first case and $r_1 \to s_1 \to r_2 \to s_2 \to r_1$ in the second. If R and S lie in different co-location groups, some placement puts r and s on different shards. Otherwise Σ does not imply the equality of the shard keys, so some finite instance M of Σ has a tuple r' of R whose witness s' under ρ has $s'[SK(S)] \neq r'[SK(R)]$. FDs and INDs are preserved under direct products [27], so $I \times M$ and $J \times M$, with pairs of values encoded as constants, satisfy Σ, and they form a modification update, since M stays fixed. In the product, (r_k, r') references (s_l, s') under ρ in a state exactly when r_k references s_l in the corresponding state of (I, J), and the second components agree because s' is the witness of r'. Hence every edge between r_k and s_l lifts to one between (r_k, r') and (s_l, s'), and the cycle lifts to $(r, r') \to (s, s') \to (r, r')$, or to $(r_1, r') \to (s_1, s') \to (r_2, r') \to (s_2, s') \to (r_1, r')$. The shard-key values of (r, r') and (s, s') differ in the second component, so some placement puts them on different shards. In both cases a cycle of $\mathcal{P}$ spans two shards under some placement, and Lem. 5.5 (1) rules out every execution by single-shard steps. □

On the running example, Thm. 5.3 admits the cached path of Ex. 4.4 as well if groups form trees, every project copies the frozen root of its tree from its group, and projects move only within their tree: sharding both relations by the root then carries the cached-path IND.

For fresh updates, the sufficiency part applies verbatim. The first case of the necessity part is a fresh update, also in the product, whose new source values are pairs with a new first component, so every IND with a mutable copy of a mutable source must carry the shard key. Neither bound is exact. An IND whose pointer and source are mutable and whose copy is frozen yields only edges from R to S under fresh updates, by the argument of the proof of Prop. 4.5, which applies to each IND separately. If it is the only IND that violates condition (4), every fresh update is executable entity by entity, so it need not carry the shard key. However, INDs can combine, for example two INDs between the same relations. Let $R(N, A, B, D, E)$ and $S(L, C)$ have the primary keys N and L, let only A, C and E be mutable, and let the INDs be $R[AD] \subseteq S[LC]$ and $R[BE] \subseteq S[LC]$, with $SK(R) = B$ and $SK(S) = L$ in one co-location group. The second IND carries the shard key, and the first has a frozen copy. Let r_1 and r_2 reference s_1 and s_2 under the second IND and s_2 and s_1 under the first, and let every value of C, D and E be 1. Consider the fresh update that gives s_1 and s_2 new values of C, gives r_1 and r_2 the matching values of E, and moves both under the first IND to an unchanged entity whose C-value is 1. Its precedence graph has the cycle $r_1 \to s_2 \to r_2 \to s_1 \to r_1$. Since r_i lies on the shard of s_i, the cycle spans two shards under every placement that separates the keys of s_1 and s_2, and the update is not executable under SK. Combinations through a third relation exist as well, so an exact condition has to follow paths of INDs. A bounded SAT search confirms Thm. 5.3 on 13,199 combinations of 280 random schemas of class $\mathcal{K}$, freezes, shard keys of one attribute and co-location groups. It finds no counterexample with up to five entities and five values wherever the condition holds, and a counterexample with at most four changed entities wherever it fails.

Insertions and deletions need no precedence graph when the INDs are acyclic.

Proof of Prop. 5.4. Let the update only insert entities, so that $I \subseteq J$, and write the relations so that S precedes R whenever Σ has an IND from R to S. Every prefix state L of this order satisfies $I \subseteq L \subseteq J$, and a subset of J satisfies every FD that J satisfies. Let t be a tuple of L in a relation R, and let $R[X] \subseteq S[Y]$ be an IND. If t is old, its witness in I lies in L. If t is new, J holds a witness s of t, and s is old or written before t, since S precedes R and $S \neq R$ by acyclicity. For deletions, the order passes through the states of the insertion from J to I in the reverse order, which writes referencing relations before referenced ones. □

Lem. 5.5 reads the precedence graph as the complete list of violations.

Proof of Lem. 5.5. Since the update is binary, an intermediate state $I \oplus T$ is legal if and only if no edge of $\mathcal{P}$ leads from an entity outside T to an entity in T. (1) Order the strongly connected components of $\mathcal{P}$ topologically and write each in one step, which touches one shard. After every step, the written entities are closed under predecessors, so the state after every step is legal. Conversely, consider the first step that writes an entity of a component that spans two shards. This step writes only part of the component, so some edge of the component leads from an unwritten entity to a written one, and the state after the step is illegal. (2) If every shard commits exactly once, an execution is an order of the shards. It is legal if and only if every edge between two shards leads from an earlier shard to a later one. Such an order exists if and only if the graph obtained by merging the entities of every shard is acyclic. □

Rounds refine model (1) by letting the transactions of one round commit in any order.

Proof of Prop. 5.7. A state is legal if and only if no edge of $\mathcal{P}$ leads from an unwritten entity to a written one, since the update is binary. Let an execution in d rounds be legal, and let $r(e)$ be the round of the transaction that writes the entity e. Consider an edge from v to u. If $r(v) > r(u)$, the state after round $r(u)$ writes u but not v. If $r(v) = r(u)$ and u and v lie in different transactions, the transaction of u may commit first. Both states are illegal, so $r(v) \leq r(u)$, with equality only if u and v share a transaction and hence a shard. Along a path that ends in e, the round therefore grows by at least one at every edge between two shards, and $d \geq r(e) \geq 1 + \lambda(e)$.

Conversely, every cycle of $\mathcal{P}$ lies within a strongly connected component and hence within one shard. Removing a cycle from a walk therefore keeps all its edges between two shards, so $\lambda(e)$ is also the largest number of such edges on a walk that ends in e. For an edge from v to u, a walk that ends in v extends to u, so $\lambda(v) \leq \lambda(u)$, and $\lambda(v) < \lambda(u)$ if the edge joins two shards. Hence v lies in the transaction of u or in an earlier round. After the transactions of the earlier rounds and any of the current round have committed, the written entities are therefore closed under predecessors, and every state is legal. □

Alg. 1 assumes that the shard map, which places every entity by its shard key, does not change during an update, and that no

constraint checked within a shard relates writes of two rounds. Such a write would wait for an earlier round to commit, whereas the algorithm executes all writes first.

To complete an update after a crash without a retry by the client (Sec. 5.3), every transaction of the first round also stores the update in an outbox on its shard, together with the tuples that it read. From this outbox, the relay commits the remaining rounds. A crash may also leave a round partly committed, since its transactions commit independently. Such a state is legal, as no edge of $\mathcal{P}$ joins two transactions of one round, and writing the round again changes nothing for the committed entities. A relay that re-applies the new tuples blindly, as Sentry does, may overwrite a later update or move a child into a parent deleted in the meantime. Our relay therefore validates what the update read. It faces the same constraints as the original rounds, and its locks protect it only from updates that respect them. Here an entity *e shares a constraint* with a tuple t of another entity if one of the two tuples, t or the tuple of e, agrees with the other on the left side of an FD or is a witness of the other under an IND. An entity shares a constraint with an entity u of an update if it shares one with the current, the old or the new tuple of u, and a relay reads an entity that is absent as absent. We call an update *fenced* if it changes an entity only under an exclusive lock and commits a new tuple only while it holds a lock on every other entity that shares a constraint with the tuple, each taken before the update read that entity. Under primary keys and INDs into them, these entities are the parents and the children of the tuple, and a witness is the entity that carries the referenced key, so a fenced update locks the parent of every child that it inserts or moves.

Proposition A.18 (Recovery). *Let Σ consist of primary keys and INDs into them, let the other updates be fenced and lead from legal states to legal states, and let a crash stop leveled commits after some round in a legal state. Let a relay commit the remaining rounds as a fenced update that, before round k commits, locks and reads the entities of round k, then reads every entity that shares a constraint with one of them, and aborts unless all are as round $k-1$ left them. Then every state after a commit is legal, and the relay completes the update unless another update has changed one of these entities.*

Comparing values suffices, since legality depends on the values alone: an entity that another update changed and restored is as good as untouched. An aborted relay leaves the update stopped after some round in a legal state, and its outbox tells the application which writes remain, so that it can plan them again on the current state or report the update as failed. The client learns the outcome from the outbox as well, since the relay commits the later rounds without it.

Proof of Prop. A.18. The crash leaves a legal state after some round i, and other updates lead from legal states to legal states, so it suffices to show that every commit of the relay does as well. A round that the relay aborts makes none of its writes visible. Otherwise, consider the commit of a round $k > i$, and let u be an entity of round k. The relay locks u exclusively before it reads the entities that share a constraint with u, and, being fenced, it holds a lock on the parents of the new tuple of u until round k has committed. Hence no other update changes these parents between the read and the commit. No other update commits a tuple of a child of u in this time either, since being fenced it would need a lock on u. A child that another update has moved away from u before the read was read as such, and one that it moves away after the read no longer shares a constraint with u. Hence, at the commit, every entity that shares a constraint with an entity of round k is as the relay read it, which is as round $k-1$ left it. An IND into a primary key is violated by a tuple and the entity that carries the referenced key. A violation after the commit that involves no entity of round k thus exists before the commit. Any other violation pairs an entity of round k with another entity of round k or with one as round $k-1$ left it, and so also exists in the state after round k of the original execution, which is legal by Prop. 5.7. Hence the state after the commit is legal. The relay overwrites only entities that it has locked and found as the crash left them, so it overwrites no change of another update. □

A pause between rounds shows every fast reader a legal state, since it orders the commits of different rounds in time.

Proof of Prop. 5.8. Times are real times, and the clocks of the writer and the reader measure durations alike. For a round i, let a_i be the time at which the last transaction of round i acknowledged its commit. Every transaction of round i commits before a_i, and every transaction of a later round k commits after $a_{k-1} + \delta \geq a_i + \delta$, since its commit is sent no earlier. The reader reads every shard s at one time t_s and sees on s exactly the transactions of s that committed before t_s (assumption 1), and $|t_s - t_{s'}| < \delta$ for all shards s and s'. If the reader sees no transaction of the update, it sees the initial instance, which is legal. Otherwise, let j be the latest round of which it sees a transaction, on a shard s'. If $j > 1$, then $t_{s'} > a_{j-1} + \delta$, and every transaction of a round $i < j$, on a shard s, committed before $a_i \leq a_{j-1} < t_{s'} - \delta < t_s$. Hence the reader sees every transaction of the rounds before j, some of round j and none of the later rounds. By the proof of Prop. 5.7, the entities that these transactions write are closed under predecessors, so the state that the reader sees is legal. □

Replicas shift the times at which a reader sees a shard by at most their lag.

Corollary A.19 (Replicas). *Let every shard be read from a replica that applies the commits of the shard in their order, at most Δ after they happen. Then Prop. 5.8 holds for every reader that reads each replica once, and all replicas within less than $\delta - \Delta$.*

Proof. A replica read at time t shows a prefix of the commits of its shard that contains every commit before $t - \Delta$ and none after t. Hence it shows exactly the transactions of its shard that committed before some time t' with $t - \Delta \leq t' \leq t$. Read times within less than $\delta - \Delta$ of each other thus yield such times t' within less than δ of each other, and the proof of Prop. 5.8 applies to them. □

Order-sufficiency judges every update alone, whereas concurrent updates interleave their writes. The precedence graph shows that concurrent updates that change disjoint entities need only merge into a legal instance.

Proposition A.20 (Concurrency). *Let updates that change pairwise disjoint sets of entities start from a common instance, and let their combination, whose changed entities are the union of theirs,*

be binary. If applying all of them yields a legal instance, then every interleaving of legal orders of the updates passes only through legal instances.

Proof of Prop. A.20. Applying all updates yields a legal instance, so their combination is an update, and let $\mathcal{P}$ be its precedence graph. Since the combination is binary, an intermediate state of the combination is legal if and only if no edge of $\mathcal{P}$ leads from an unwritten entity to a written one. Applying a single update alone yields its final instance, which is legal and is the intermediate state of the combination that writes exactly the changed entities of this update. Hence no edge of $\mathcal{P}$ enters the changed entities of an update from those of another. A state of an interleaving writes a prefix of the order of every update. An edge into a written entity u therefore starts at an entity of the same update, and since the prefix states of that update alone are legal, it starts within the written prefix. □

Updates that do not merge into a legal instance, such as deleting a group g while another update inserts a project into it, leave an orphan under ordered commits and under one distributed transaction alike. Such updates must lock the shared parent row, as the relay does. In the example, leveled commits hold the lock on g in their open transactions: the deleting update locks g in the transaction of its last round before it reads the projects of g, and the inserting update locks g for sharing in a transaction that commits after the project.

These locks can deadlock across shards unnoticed by any shard, since a later round waits for an earlier one in the application. They cause no such deadlocks if their requests fail instead of waiting.

Lemma A.21 (No hidden deadlocks). *If every lock request of leveled commits, for a write, a fence or a validation, fails instead of waiting, and an update whose request fails aborts its open transactions, then no update waits for a lock while it holds one, and no deadlock arises.*

Proof. A transaction of leveled commits acquires locks only while it executes its statements, never waits for a lock, and releases its locks when it commits or aborts. Afterwards, an update waits only for the commits of its own earlier rounds, and a commit acquires no lock. A deadlock is a cycle of updates in which each waits for a lock that the next one holds, so none arises. □

Retries with randomized backoff keep updates that fail on the same lock from failing again in step, but they guarantee no progress: under heavy contention most attempts fail (App. B.2), and a relay whose locks fail stops the update as an abort does.

B FURTHER EXPERIMENTAL RESULTS

This appendix reports the results of Sec. 6 that the argument of the body does not need, with one subsection for each of Secs. 6.2 to 6.4.

B.1 Results for Sec. 6.2

The answer to (E1) rests on our labels of GitLab and Sentry and, for the ten further applications, on the analyzer and the blind labels. The cross-database write templates of GitLab are the 15 transactions that it exempts from its guard against such writes and the deletions of 17 parents whose foreign keys cross databases. Those of Sentry are its 42 outbox replications and 23 synchronous remote writes.

Table 1: The Analyzer against the Hand Labels

	GitLab and Sentry (our labels)	Ten applications (blind labels)
Templates labeled	97	135
Found by the analyzer	72	135
Labels agreed	57	79
Unconstrained, analyzer says order	1	40
Order, analyzer says unconstrained	9	0
Atomic-visibility labels missed	4	16
Atomic-visibility verdicts right	11 of 12	5 of 5

The results below show which constraints the templates maintain, which writes make order matter, and how far the labels and the analyzer agree.

Coverage of the exact classes. The exact classes of Sec. 4 cover the constraints of 63 of the 97 templates of GitLab and Sentry. Foreign keys alone, which class $\mathcal{A}$ covers, constrain 25 of the 65 templates of Sentry and 22 of the 32 of GitLab. Another 16 of Sentry add copies of referenced columns over one hop, which class $\mathcal{K}$ covers together with the unique keys of Prop. A.12. Five templates are unconstrained, and the others maintain conditional INDs, derived values or copies over two hops.

Writes of the ten applications. In the ten further applications, write order matters mostly for templates that mix modifications with insertions or deletions. Of the 4,998 templates that need only order, 681 only insert or delete rows, 286 move pointers along references, and the other 4,031 modify further attributes next to insertions or deletions. About 1,070 of the latter maintain derived values, which need atomic visibility if they must stay exact, and so lower the share of order to 78% (Sec. 6.2).

Hand labels. Hand labels check the analyzer, ours for the 97 templates of GitLab and Sentry and blind ones for 135 templates of the ten applications (Sec. 6.2). The second annotator of Sec. 6.1 doubts three of our labels only under concurrent updates, which the locks of Sec. 5.3 serialize, so we kept our labels, and the artifact holds both sheets. Of the 67 blind labels of atomic visibility, 46 hold only for exact derived values and 21 for copies. The 82% of Sec. 6.2 reads the 46 as order, since derived values may lag, and weighs each stratum of the sample, an application and a verdict of the analyzer, by its number of templates.

The analyzer. Table 1 compares the analyzer with both sets of labels. Its verdicts of atomic visibility are almost always right, whereas it misses copies that it cannot parse and disagrees with the labels most often on whether a template is unconstrained or needs only order. Of the 21 copies among the blind labels, it finds 5. The copies that it misses are written by dictionary updates, bulk updates, computed values, polymorphic references and many-to-many mirrors. Hence the share of order that the analyzer reports is too high, and Sec. 6.2 estimates it from the blind labels instead.

B.2 Results for Sec. 6.3

This subsection explains why leveled commits take less time than 2PC, measures a faster and a safer variant of them, and gives the costs of their recovery and locks. In Figs. 6a to 6e, each value is a geometric mean over the three templates of Sec. 6.1 and three repetitions, and its 95% interval lies within 12% of it.

Access path, routing and placement. The access path of single-shard transactions explains most of the gain of leveled commits over 2PC (Sec. 6.3). On Citus with one client, the phase that executes the statements before any commit holds 78% of the gap to the fastest 2PC at 8 rows and 95% at 128 rows. Routing through the coordinator forfeits part of this gain: routed through the coordinator of Citus, leveled commits still take about half the time of 2PC at 128 rows with one client, but not at 8 rows behind the 1 ms proxies, and with 64 clients the coordinator becomes the bottleneck. Placement changes little: with every node on cores of its own and 0.5 ms added to every path between nodes, leveled commits still take at most 0.6 of the time of 2PC and reach at least twice its throughput on both engines.

An eager first round. Executing every write before the first round commits costs one round trip, which lets a logical failure abort an update before any write is visible (Sec. 5.3). On two or three rows, ordered commits of one row each commit every row in the message that executes it and take up to a quarter less time with one client, but a row that fails leaves the earlier rows visible. Where no write can fail logically, an *eager first round* commits in the message that executes it and saves this round trip. It lowers the latency of leveled commits by 7 to 12% with one client and by 22 to 28% behind the 1 ms proxies, which makes them the fastest way of writing with one client at every size.

Ordered 2PC. An update that must never stop between rounds needs atomicity on top of order, which costs more, though less than the 2PC of the engines. An ordered 2PC, which prepares all transactions of leveled commits before the first round commits (Sec. 5.3), takes 6 to 26% longer with one client and up to halves their throughput with 64 clients, yet it stays faster than the fastest 2PC. Indeed, each prepare flushes a log, for which clients contend. Leveled commits thus trade atomicity for this margin.

Recovery. Without atomicity, a crash may stop leveled commits between rounds, and completing the update without the client costs little. An outbox, one row in every transaction of the first round, raises the latency of leveled commits by at most a fifth and leaves them faster than 2PC at every point of Figs. 6a to 6e. When a proxy dropped a commit message to one Citus worker every half second, one 2PC reported success for every update, but 37 projects stayed orphans until the periodic recovery of Citus completed the lost commits, up to 38 s later at its default interval of one minute. A shorter interval narrows this window but cannot close it. Leveled commits with an outbox left no orphan, since a lost message stops them between rounds, where the state is legal, and the client or the relay then completes the update. Validation matters once another update intervenes. In 3,000 recoveries that raced with a conflicting update, a relay that re-applied the new tuples left 653 orphans and lost 381 updates. The relay of Prop. A.18 left none, completing two thirds of the updates and stopping the others in a legal state.

Locks and deadlocks. Recovery handles failures, whereas concurrent updates that do not merge into a legal instance need the locks of Sec. 5.3. When clients added projects to groups that other clients merged, 24% of the merges left an orphan under leveled commits and 68% under one 2PC. With the locks, no merge did, merges took as long as before, and additions took 0.85 instead of 0.71 ms. Lock requests that fail instead of waiting keep these locks free of hidden deadlocks (Lem. A.21), at the price of retries. When

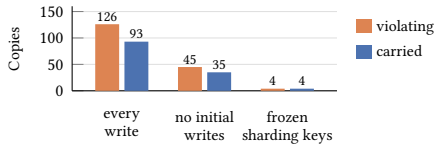


Figure 9: Copies of GitLab Violating Condition (4) of Thm. 4.3, and Those Carried by Co-Location

64 clients moved projects between hot groups, writes that waited closed cycles of waits across workers, which only a lock timeout broke, whereas writes that failed closed none and sustained 6,440 updates per second, although 63.5% of their attempts failed and their 99th percentile latency reached 184 ms. One 2PC that waited for the same locks sustained at most 35, since only the distributed deadlock detection of Citus broke its cycles. With a lock timeout or no-wait locks it would fail fast like leveled commits, a configuration that we did not run. Locking the rows in one global order avoids the cycles as well and sustained 3,871 updates per second. With 8 or 16 clients under milder skew, waits bounded by a timeout of 100 ms were faster than failing writes, so failing writes pay off only under heavy contention.

B.3 Results for Sec. 6.4

This subsection confirms the verdicts and freezes of (E3) and reports further gains of the design for (E4).

Deciders and Freeze. Bounded search confirms the verdict of every decider on all 106,270 random pairs of schema and freeze that we tried, with a direct construction of counterexamples for 437 rejected pairs of class $\mathcal{K}$ that need 6 to 12 entities. Alg. A.1 is optimal on bipartite conflict graphs, where it computes the minimum cut of Sec. 6.4, and on graphs with odd cycles its 2-approximation stays within 1% of the linear-programming lower bound.

Copies of GitLab. Fig. 9 shows how many copies of GitLab violate condition (4) of Thm. 4.3 as more writes count as modifications (Sec. 6.4). The left bar counts every write, even one that initializes a new row, the middle bar leaves out such initial writes, and the right bar also freezes the sharding keys.

FDs of GitLab. Frozen sharding keys settle the FDs of GitLab as they settle its copies in Fig. 9. GitLab has 671 FDs whose left sides are no keys, and all of them copy frozen sharding keys, so no template modifies their right sides. Hence, by the argument for FDs together with class $\mathcal{K}$ in App. A.3, no FD of GitLab needs atomic visibility for fresh updates.

Unique keys under sharding. A shard key that makes updates executable may still split a unique key, which a shard then cannot check. In GitLab, 1,040 of the 1,127 tables have a single-column shard key that makes every fresh update executable and that every unique key of the table determines through copies, so that a shard checks the key from local data [35]. The other 87 tables hold globally unique names, paths and tokens, which need a check across shards under any placement. The rule of Citus [20], which requires the shard key inside the primary key and every unique key, admits a single-column shard key for only 527 tables.

Optimal freezing. Besides shard keys, the design chooses the attributes to freeze, and an optimal freeze pays off once templates

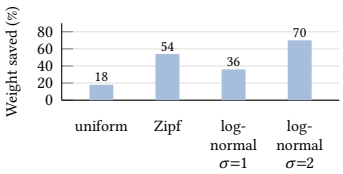


Figure 10: Weight Saved by the Optimal Freeze over Freezing Every Copied Source, by Frequency Distribution

modify attributes at different frequencies. On the 179 INDs of GitLab that copy shard keys and that class $\mathcal{K}$ covers, the conflict graph consists of stars around the sources, so under unit weights freezing every copied source is optimal. Since GitLab does not record update frequencies, we draw them at random. Fig. 10 shows that the more skewed they are, the more weight the optimal freeze saves over this rule. It saves a median 18% under uniform frequencies and 36 to 70% under log-normal ones. Indeed, a star whose copies weigh less than its source is covered by them, and skew makes such stars common.